

**Advance Database Management
Coursework**

**Higher National Diploma in Software Engineering
HDSE242F**



**School of Computing
National Institute of Business Management
Galle**

National Institute of Business Management
Higher National Diploma in Software Engineering
GAHDSE 242F
ADBM Coursework Report

Submitted By

GAHDSE242F-003	Nanayakkara B G S M
GAHDSE242F-004	Anuththara H L L T
GAHDSE242F-005	Pramudith D S
GAHDSE242F-010	Susiripala M R S
GAHDSE242F-011	Leewanage P J

Supervised By

Mr. Rukshan Manchanayake

Contents

Urban Foods.....	8
Use Case.....	8
ER Diagram.....	9
Strengths and Weaknesses of using RDBMS versus a NoSQL Database.....	10
Definitions.....	10
RDBMS (Relational Database Management System):	10
Strengths and Weaknesses of RDBMS vs NoSQL (MongoDB).....	11
ORACLE Relational Database for Urban Food E-Commerce Platform	16
Data Definition Language (DDL) Commands for Creating Database	16
DATA CONVERSION IN PL SQL	21
Data Manipulation Language (DML) Commands	23
JOINS AND LOOPS.....	29
IF THEN ELSE EXECUTIONS.....	31
LOOPS IN PL SQL.....	32
USING EXPLICIT CURSORS	35
EXCEPTIONS.....	39
PROCEDURES.....	41
DATA DEFINITION LANGUAGE.....	44
NoSQL (MongoDB).....	46
References	56

Table Of Figures

Figure 1-ER Diagram.....	9
Figure 2-Supplier Table.....	16
Figure 3-Customer Table.....	16
Figure 4-Restaurant Table	16
Figure 5-Products Table	17
Figure 6-Orders Table	17
Figure 7-Order_Details Table	17
Figure 8-Payments Table.....	18
Figure 9-Deliveries Table.....	18
Figure 10-Reviews Table	18
Figure 11-Feedback Table	18
Figure 12-Retrieving data from product table to variables	19
Figure 13-Using bind variables to retrieve data from product table	19
Figure 14-Retrieving multiple rows from product table	20
Figure 15-Explicit conversion using built in function	21
Figure 16-Explicit conversion using built in function(results)	21
Figure 17-Implicit conversion to convert varchar into number dynamically	22
Figure 18-Implicit conversion to convert varchar into number dynamically(results)	22
Figure 19-Insert Data to Product table using sequence	23
Figure 20-Updating data of product table	23
Figure 21-Delete data from product table	23
Figure 22-Group by commands to view maximum, minimum, total, average and count number of prices in each category(results)	24
Figure 23-Group by commands to view maximum, minimum, total, average and count number of prices in each category	24
Figure 24-Group by commands to find suppliers who supplier more than 3 various products using having clause	25
Figure 25-Sub query using IN to get the suppliers who supplied product stock more than 100	25
Figure 26-Sub query using operators to get product details which price is higher than average product price	26
Figure 27-Sub query using ANY and ALL to retrieve product details	27

Figure 28-Sub query using ANY and ALL to retrieve product details	27
Figure 29-Merge two tables Product_update and Product	28
Figure 30-Merge two tables Product_update and Product	28
Figure 31-Inner join customer and order table	29
Figure 32-Left Outer join with product and suppliers to view all product data with related supplier details	29
Figure 33-Right Outer join with suppliers and product to view all suppliers with their product details	30
Figure 34-Full Outer join to view all suppliers and product details even products without suppliers and suppliers without products are included	30
Figure 35-Check order status using if condition and update to next status-	31
Figure 36-Check order status using if condition and update to next status	31
Figure 37-Using a loop to fetch product tables multiple rows.....	32
Figure 38-Using a loop to fetch product tables multiple rows.....	32
Figure 39-Using while Loop to fetch product tables details	33
Figure 40-Using while Loop to fetch product tables details	33
Figure 41-Using for loop to retrieve product table details.....	34
Figure 42-Using GOTO label to find expensive product which is higher than 10.0	34
Figure 43-Using GOTO label to find expensive product which is higher than 10.0	34
Figure 44-Using cursor to fetch a single row from product table	35
Figure 45-Using cursor to fetch a single row from product table(results).....	35
Figure 46-Using cursor to fetch multiples row from product table	35
Figure 47-Using cursor to fetch multiples row from product table(results)	36
Figure 48-Parameterized cursor fetch products total price from different categories	36
Figure 49-Parameterized cursor fetch products total price from different categories(results)	36
Figure 50-Cursor with FOR UPDATE to update to increase product price by 10 percent.....	37
Figure 51-Cursor with FOR UPDATE to update to increase product price by 10 percent(results)	37
Figure 52-Cursor with sub query.	37
Figure 53-Cursor with sub query.(results)	38
Figure 54-Pre-defined oracle server errors.	39
Figure 55-Pre-defined oracle server errors.(results)	39
Figure 56-too many rows exception for fetching multiple row values to single variable	39
Figure 57-Non-Pre-defined Exception.....	40

Figure 58-User defined exception to avoid inserting negative values for price	40
Figure 59-User defined exception to avoid inserting negative values for price(results)	40
Figure 60-Procedure to retrieve all products.....	41
Figure 61-Parameterized procedure to retrieve products by category	41
Figure 62-Use IN and OUT parameters in a procedure to retrieve product stock for each category	42
Figure 63-Use IN and OUT parameters in a procedure to retrieve product stock for each category(results).....	42
Figure 64-Function to return number of customers	43
Figure 65-Function to return number of customers(results)	43
Figure 66-Parameterized function to get stock count by product id	43
Figure 67-Create users	44
Figure 68-Grant privileges	44
Figure 69-Create table shashimalnew schema	44
Figure 70-Grant shashimalnew user my table privileges to lasindu user	44
Figure 71-Grant specific column privileges to lasindu user	44
Figure 72-Create User Roles.....	45
Figure 73-Grant create table Privileges to COADMIN Role with admin option	45
Figure 74-Grant select privilege of mytable in shashimalnew schema to COADMIN role	45
Figure 75-Assign user manager role to lasindu user.....	45
Figure 76-Assign User manager role to coadmin role	45
Figure 77-Insert many for reviews.....	49
Figure 78-View reviews	50
Figure 79-Insert one for feedbacks	50
Figure 80-Find Rating that Rating number is 4	51
Figure 81-Find rating that Rating numbers are 3 and 4	51
Figure 82-Find ratings less than 4.....	52
Figure 83-Find reviews where the rating is less than 4 or the review text contains "good"	52
Figure 84-Find reviews where the rating is greater than 2 and the review text contains "good"	53
Figure 85-Update One feedback document	53
Figure 86-Update One feedback document	54
Figure 87-Replace One document from reviews	55
Figure 88-Delete one feedback document where id is 8	55

Figure 89-Delete Many which is rating number is one.....55

Urban Foods

Urban food is a systematic approach for peoples to satisfy their need in food and thirst. In order to satisfy those needs the urban food platform has Shaked hands with multiple restaurants in the area to provide customers with their needs.

Below is the Database integration for Urban Foods platform and the Use Case scenario based on how the system/database operates.

Use Case

Urban food is a food ordering network that is established to provide foods and drinks to customers over the country. The network has several restaurants and here we are designing the database for a single restaurant and based on that we can improvise and establish the whole database for all the restaurants.

The network has only one restaurant at the moment and there is a one-to-many relationship between the restaurants and the products. In order to identify each restaurant, we have provided restaurantId as the primary key of the restaurant entity and for identifying the products we have used productsId as the primary key. The suppliers and the products have a one-to-many relationship as one supplier can deliver several products and to identify the suppliers, we have used supplierId as the primary key of suppliers entity.

From products we have one to many relationships for both review and orderDetails as one product can have many reviews and orderDetails and to identify those we established reviewId and orderdetailsId as their primary keys.

Each customer can provide several feedbacks and reviews and each customer also can put several orders. To identify each customer, we have customerId and to identify reviews we have review Id and to identify feedbacks we have feedbackId.

Each order can have several orderdetails and one orderdetails is attached to only one order and one order can have only one payment and delivery. To identify payment, we have paymentId as the primary key and to identify each delivery we have deliveryId.

ER Diagram

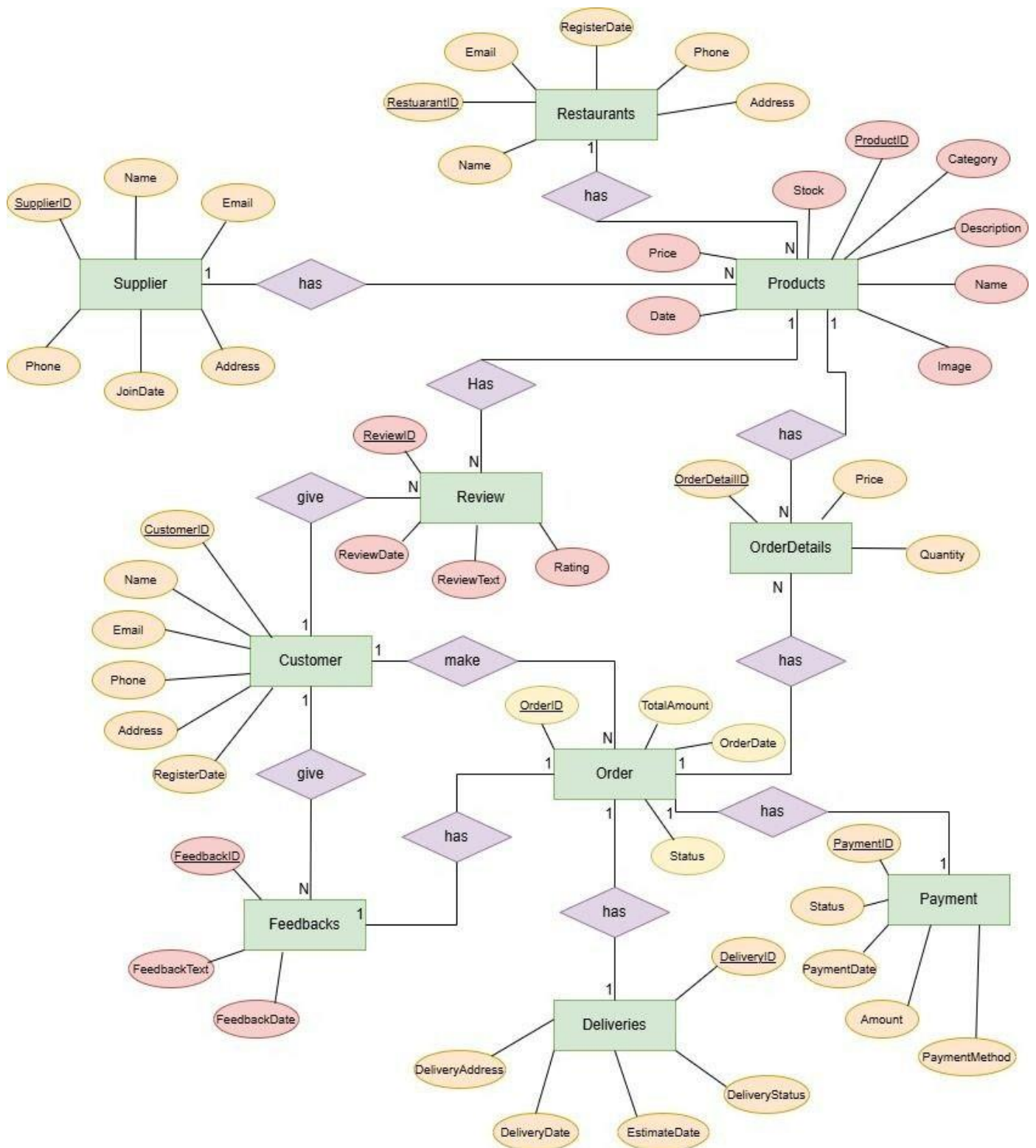


Figure 1-ER Diagram

Strengths and Weaknesses of using RDBMS versus a NoSQL Database.

Definitions

RDBMS (Relational Database Management System):

The basis of RDBMS is relational model. It follows structured approach in moving, retrieving, and storing data into a database using schemas, tables and SQL.

An RDBMS can be distinguished between Logical and Physical operations.

In Logical type operations RDBMS specifies what content is required and in Physical type operations RDBMS determines how operation should be carried out.

NoSQL Database:

NoSQL databases are non-relational database management systems

It is used in large scale data storing needs which uses distributed data stores.

NoSQL Databases use schema-less approach which allows flexible and scalable data storage.

Comparison of RDBMS and MongoDB

Feature	RDBMS	MongoDB
Database Structure	Database	Database
Table Equivalent	Table	Collection
Row Equivalent	Tuple/Row	Document
Column Equivalent	Column	Field
Joins	Table Join	Embedded Documents
Primary Key	Primary Key	Primary Key (_id provided by MongoDB itself)

Strengths and Weaknesses of RDBMS vs NoSQL (MongoDB)

Use Case	RDBMS Strengths	RDBMS Weaknesses	NoSQL Strengths	NoSQL Weaknesses	Best Choice
Product Catalog Management	The implementation of foreign keys established structured data relationships which join different entities in the database. Supports Atomicity, Consistency, Isolation, Durability	The performance suffers when complex joins occur because the catalog expands in size. The process of changing the database schema to accommodate new product attributes requires database migrations.	Products with new attributes can be added effortlessly through schema flexibility. The system provides high read/write data processing capabilities that support heterogeneous data structure formats.	When relationship strength is weak data duplication becomes more likely to happen. Certain complex relationship queries need supplementary processing for execution.	NoSQL for flexibility. RDBMS if strict integrity is needed.

Use Case	RDBMS Strengths	RDBMS Weaknesses	NoSQL Strengths	NoSQL Weaknesses	Best Choice
User Management and Authentication	When relationship strength is weak data duplication becomes more likely to happen. Certain complex relationship queries need supplementary processing for execution.	Region to region scaling of user accounts might need the complex configuration of sharing. ,	Supports high traffic and allows flexible metadata storage	Data inconsistency results from the absence of built-in relational constraints in these systems. Security attributes of NoSQL databases show decreased development compared to traditional RDBMS systems.	RDBMS for security & consistency. NoSQL for scalability.

Use Case	RDBMS Strengths	RDBMS Weaknesses	NoSQL Strengths	NoSQL Weaknesses	Best Choice
Order Processing and Transactions	The ACID standards ensure both orders and customer payments are handled properly. The system maintains complete transaction consistency to stop payment errors including duplicates.	High transaction traffic can slow down performance.	Works well in distributed systems	Operations critical to payment processing can encounter issues caused by eventual consistency. The level of ACID compliance in transactions varies depending on the chosen NoSQL system type.	RDBMS for transactions & payments. NoSQL for event-based order logging.

Use Case	RDBMS Strengths	RDBMS Weaknesses	NoSQL Strengths	NoSQL Weaknesses	Best Choice
Inventory Management	Stock consistency operates in real-time through the integration of constraints and transactions. The system enables multi-user concurrency control which prevents stock from being oversold to various users at once.	The system becomes slow to perform inventory updates at numerous distributed locations.	Fast stock updates possess the capability to process extremely large transaction volumes quickly. The distributed design enables simple expansion of global inventory management systems.	Stock updates through this system handle large numbers of transactions with high speed. The distributed design of the system enables simple expansion capabilities for managing inventory at various locations worldwide.	NoSQL (Redis) for real-time caching, RDBMS for long-term accuracy.
Customer Reviews and Ratings	The storage system maintains proper relationships between users and products. The capability to perform	Schema changes require database modifications.	The system supports scalability and demonstrates high user interaction potential.	Aggregations (such as average ratings) may be slower than SQL.	NoSQL (Document Store) for flexible, scalable reviews.

Use Case	RDBMS Strengths	RDBMS Weaknesses	NoSQL Strengths	NoSQL Weaknesses	Best Choice
	powerful review analysis is made possible through SQL query applications. SQL supports analytics.		Has the ability to manage complex data structures including varied review content and image and video assessment elements.		
Session Management and Caching	Not ideal for session data handling.	High latency in handling session data at scale.	Scales easily across multiple servers.	periodic backup needed. like foreign keys)	NoSQL (Redis, Memcached) for fast session caching.
Reporting and Analytics	SQL-based analytics make reporting simple and ensures data accuracy.	Not suited for handling large datasets. performance decrease with scaling	Can handle large scale datasets efficiently (ideal for IoT, social media, google etc.).	Complex analytics can be harder compared to SQL-based RDBMS	RDBMS for structured reports, NoSQL for real-time analytics.

ORACLE Relational Database for Urban Food E-Commerce Platform

Data Definition Language (DDL) Commands for Creating Database

- Supplier Table – Table will use to store all the supplier's data

```
-- Create Suppliers Table
CREATE TABLE Suppliers (
    SupplierID NUMBER PRIMARY KEY,
    Name VARCHAR2(255) NOT NULL,
    Email VARCHAR2(255) UNIQUE NOT NULL,
    Phone VARCHAR2(15),
    Address CLOB,
    JoinDate TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Figure 2-Supplier Table

- Customer Table – Table will use to store all the Customer data

```
-- Create Customers Table
CREATE TABLE Customers (
    CustomerID NUMBER PRIMARY KEY,
    Name VARCHAR2(255) NOT NULL,
    Email VARCHAR2(255) UNIQUE NOT NULL,
    Phone VARCHAR2(15),
    Address CLOB,
    RegisterDate TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Figure 3-Customer Table

- Restaurant Table - Table will be used to store all the Restaurants registered in Urban Food E-Commerce Platform

```
-- Create Restaurants Table
CREATE TABLE Restaurants (
    RestaurantID NUMBER PRIMARY KEY,
    Name VARCHAR2(255) NOT NULL,
    Email VARCHAR2(255) UNIQUE NOT NULL,
    Phone VARCHAR2(15),
    Address CLOB,
    RegisterDate TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Figure 4-Restaurant Table

- Products Table – Table will be used to store details of products customers can purchase

```
CREATE TABLE Products (
  ProductID NUMBER PRIMARY KEY,
  SupplierID NUMBER NULL,
  RestaurantID NUMBER NULL,
  Name VARCHAR2(255) NOT NULL,
  Description CLOB,
  Price NUMBER(10,2) NOT NULL,
  Stock NUMBER DEFAULT 0 NOT NULL,
  Category VARCHAR2(50) CHECK (Category IN ('Fruits', 'Vegetables', 'Dairy', 'Baked Goods', 'Crafts')),
  ImageURL VARCHAR2(500),
  CreatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_product_supplier FOREIGN KEY (SupplierID) REFERENCES Suppliers(SupplierID) ON DELETE CASCADE,
  CONSTRAINT fk_product_restaurant FOREIGN KEY (RestaurantID) REFERENCES Restaurants(RestaurantID) ON DELETE CASCADE,
  CONSTRAINT chk_supplier_or_restaurant CHECK (
    (SupplierID IS NOT NULL AND RestaurantID IS NULL) OR
    (RestaurantID IS NOT NULL AND SupplierID IS NULL)
  )
);
```

Figure 5-Products Table

- Orders Table - Table will be used to store all the Order details of Customers

```
-- Create Orders Table
CREATE TABLE Orders (
  OrderID NUMBER PRIMARY KEY,
  CustomerID NUMBER,
  OrderDate TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  Status VARCHAR2(50) DEFAULT 'Pending' CHECK (Status IN ('Pending', 'Processing', 'Shipped', 'Delivered', 'Cancelled')),
  TotalAmount NUMBER(10,2) NOT NULL,
  CONSTRAINT fk_order_customer FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID) ON DELETE CASCADE
);
```

Figure 6-Orders Table

- Order_Details Table - Table will be used to store all the product details include in specific orders

```
-- Create OrderDetails Table
CREATE TABLE OrderDetails (
  OrderDetailID NUMBER PRIMARY KEY,
  OrderID NUMBER,
  ProductID NUMBER,
  Quantity NUMBER NOT NULL,
  Price NUMBER(10,2) NOT NULL,
  CONSTRAINT fk_order_details_order FOREIGN KEY (OrderID) REFERENCES Orders(OrderID) ON DELETE CASCADE,
  CONSTRAINT fk_order_details_product FOREIGN KEY (ProductID) REFERENCES Products(ProductID) ON DELETE CASCADE
);
```

Figure 7-Order_Details Table

- Payments Table – Table will be used to store all payment details related to orders

```
-- Create Payments Table
CREATE TABLE Payments (
    PaymentID NUMBER PRIMARY KEY,
    OrderID NUMBER,
    PaymentDate TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    Amount NUMBER(10,2) NOT NULL,
    PaymentMethod VARCHAR2(50) CHECK (PaymentMethod IN ('Credit Card', 'Debit Card', 'PayPal', 'Cash on Delivery')),
    Status VARCHAR2(50) DEFAULT 'Pending' CHECK (Status IN ('Pending', 'Completed', 'Failed')),
    CONSTRAINT fk_payment_order FOREIGN KEY (OrderID) REFERENCES Orders(OrderID) ON DELETE CASCADE
);
```

Figure 8-Payments Table

- Deliveries Table -will be used to store delivery details of orders

```
-- Create Deliveries Table
CREATE TABLE Deliveries (
    DeliveryID NUMBER PRIMARY KEY,
    OrderID NUMBER,
    DeliveryAddress CLOB NOT NULL,
    DeliveryStatus VARCHAR2(50) DEFAULT 'Pending' CHECK (DeliveryStatus IN ('Pending', 'Out for Delivery', 'Delivered')),
    EstimatedDeliveryDate DATE,
    DeliveredDate DATE NULL,
    CONSTRAINT fk_delivery_order FOREIGN KEY (OrderID) REFERENCES Orders(OrderID) ON DELETE CASCADE
);
```

Figure 9-Deliveries Table

- Reviews Table – will be used to store reviews giving by customers for products

```
-- Create Reviews Table
CREATE TABLE Reviews (
    ReviewID NUMBER PRIMARY KEY,
    CustomerID NUMBER,
    ProductID NUMBER,
    Rating NUMBER CHECK (Rating BETWEEN 1 AND 5),
    ReviewText CLOB,
    ReviewDate TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_review_customer FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID) ON DELETE CASCADE,
    CONSTRAINT fk_review_product FOREIGN KEY (ProductID) REFERENCES Products(ProductID) ON DELETE CASCADE
);
```

Figure 10-Reviews Table

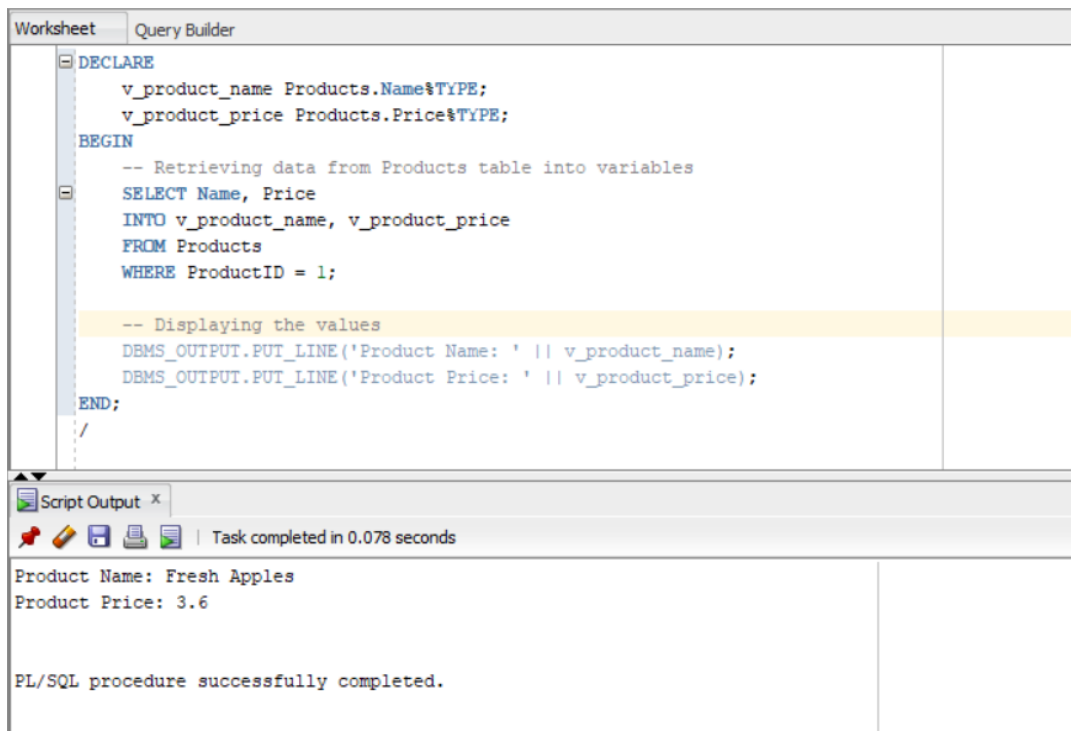
- Feedback Table - will be used to store feedback details given by customers regarding orders

```
-- Create Feedbacks Table
CREATE TABLE Feedbacks (
    FeedbackID NUMBER PRIMARY KEY,
    OrderID NUMBER UNIQUE,
    CustomerID NUMBER,
    FeedbackText CLOB NOT NULL,
    FeedbackDate TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_feedback_order FOREIGN KEY (OrderID) REFERENCES Orders(OrderID) ON DELETE CASCADE,
    CONSTRAINT fk_feedback_customer FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID) ON DELETE CASCADE
);
```

Figure 11-Feedback Table

Executing PL SQL Blocks

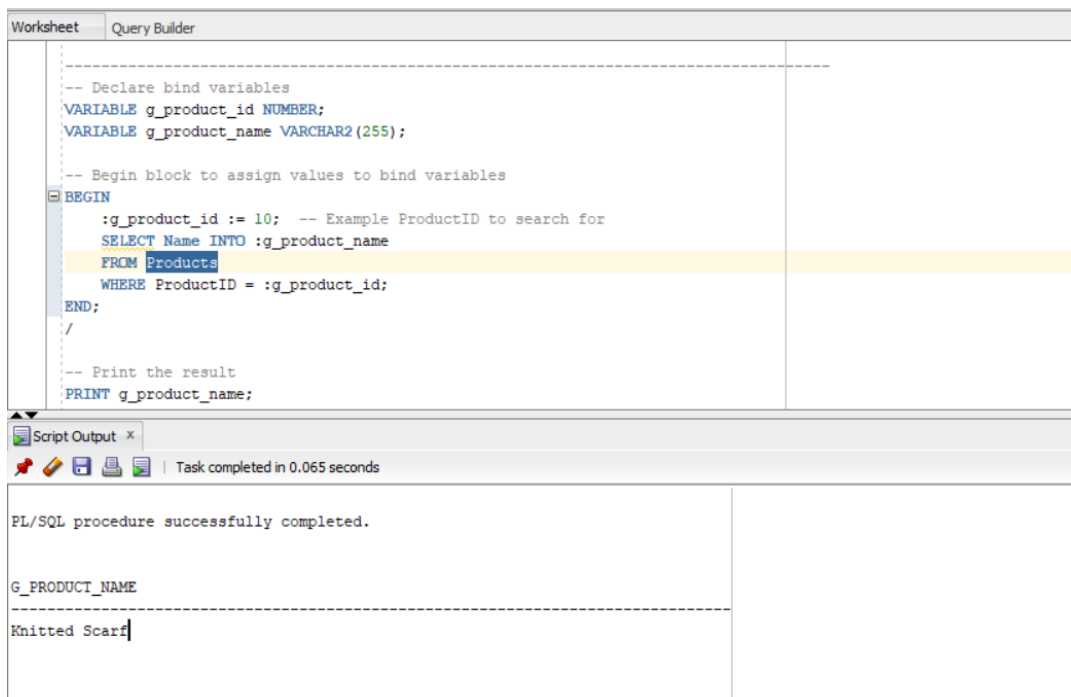
- Retrieving data from product table to variables



```
Worksheet | Query Builder
+-----+
| DECLARE |
| v_product_name Products.Name%TYPE; |
| v_product_price Products.Price%TYPE; |
| BEGIN |
| -- Retrieving data from Products table into variables |
| SELECT Name, Price |
| INTO v_product_name, v_product_price |
| FROM Products |
| WHERE ProductID = 1; |
| |
| -- Displaying the values |
| DBMS_OUTPUT.PUT_LINE('Product Name: ' || v_product_name); |
| DBMS_OUTPUT.PUT_LINE('Product Price: ' || v_product_price); |
| END; |
| / |
+-----+
Script Output x
+-----+
Task completed in 0.078 seconds
Product Name: Fresh Apples
Product Price: 3.6
PL/SQL procedure successfully completed.
```

Figure 12-Retrieving data from product table to variables

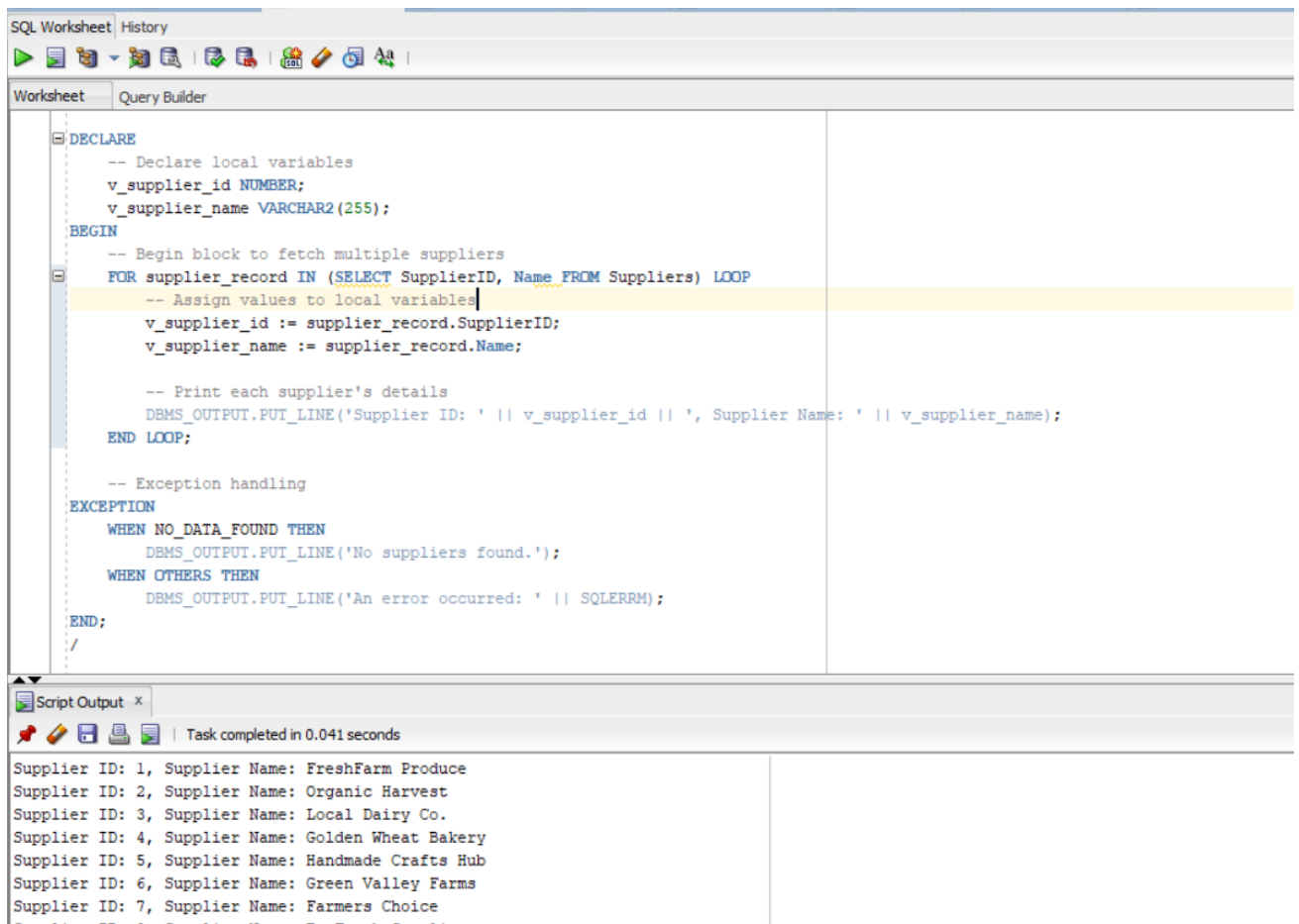
- Using bind variables to retrieve data from product table



```
Worksheet | Query Builder
+-----+
| -- Declare bind variables |
| VARIABLE g_product_id NUMBER; |
| VARIABLE g_product_name VARCHAR2(255); |
| |
| -- Begin block to assign values to bind variables |
| BEGIN |
| :g_product_id := 10; -- Example ProductID to search for |
| SELECT Name INTO :g_product_name |
| FROM Products |
| WHERE ProductID = :g_product_id; |
| END; |
| / |
| |
| -- Print the result |
| PRINT g_product_name; |
+-----+
Script Output x
+-----+
Task completed in 0.065 seconds
PL/SQL procedure successfully completed.
G_PRODUCT_NAME
-----
Knitted Scarf
```

Figure 13-Using bind variables to retrieve data from product table

- Retrieving multiple rows from product table



```

SQL Worksheet | History
Worksheet | Query Builder

DECLARE
    -- Declare local variables
    v_supplier_id NUMBER;
    v_supplier_name VARCHAR2(255);
BEGIN
    -- Begin block to fetch multiple suppliers
    FOR supplier_record IN (SELECT SupplierID, Name FROM Suppliers) LOOP
        -- Assign values to local variables
        v_supplier_id := supplier_record.SupplierID;
        v_supplier_name := supplier_record.Name;

        -- Print each supplier's details
        DBMS_OUTPUT.PUT_LINE('Supplier ID: ' || v_supplier_id || ', Supplier Name: ' || v_supplier_name);
    END LOOP;

    -- Exception handling
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('No suppliers found.');
```

```

    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('An error occurred: ' || SQLERRM);
END;
/

Script Output x
Task completed in 0.041 seconds

Supplier ID: 1, Supplier Name: FreshFarm Produce
Supplier ID: 2, Supplier Name: Organic Harvest
Supplier ID: 3, Supplier Name: Local Dairy Co.
Supplier ID: 4, Supplier Name: Golden Wheat Bakery
Supplier ID: 5, Supplier Name: Handmade Crafts Hub
Supplier ID: 6, Supplier Name: Green Valley Farms
Supplier ID: 7, Supplier Name: Farmers Choice

```

Figure 14-Retrieving multiple rows from product table

DATA CONVERSION IN PL SQL

Explicit conversion using built in function

```
DECLARE
    v_order_id NUMBER;
    v_customer_id NUMBER;
    v_order_date VARCHAR2(20); -- Store the date as a string (converted from DATE)
    v_total_amount VARCHAR2(20); -- Store the amount as a string (converted from NUMBER)
    v_status VARCHAR2(50);
    v_order_timestamp TIMESTAMP; -- Store the timestamp (converted from DATE)
BEGIN
    FOR order_record IN (
        SELECT OrderID, CustomerID, OrderDate, TotalAmount, Status
        FROM Orders
    ) LOOP
        -- Convert OrderDate to string using TO_CHAR (Date to String)
        v_order_date := TO_CHAR(order_record.OrderDate, 'DD-MON-YYYY');

        -- Convert TotalAmount to string with formatting using TO_CHAR (Number to String)
        v_total_amount := TO_CHAR(order_record.TotalAmount, '9999.99');

        -- Convert Status (using TO_CHAR for string values)
        v_status := TO_CHAR(order_record.Status);

        -- Convert OrderDate to Timestamp using TO_TIMESTAMP (Date to Timestamp)
        v_order_timestamp := TO_TIMESTAMP(TO_CHAR(order_record.OrderDate, 'YYYY-MM-DD HH24:MI:SS'), 'YYYY-MM-DD HH24:MI:SS');

        -- Output the converted values
        DBMS_OUTPUT.PUT_LINE('Order ID: ' || order_record.OrderID ||
            ', Customer ID: ' || order_record.CustomerID ||
            ', Order Date: ' || v_order_date ||
            ', Total Amount: ' || v_total_amount ||
            ', Status: ' || v_status ||
            ', Order Timestamp: ' || TO_CHAR(v_order_timestamp, 'DD-MON-YYYY HH24:MI:SS'));
    END LOOP;

    -- Exception handling
EXCEPTION
```

Figure 15-Explicit conversion using built in function

Script Output x

Task completed in 0.062 seconds

Order ID: 1, Customer ID: 1, Order Date: 29-MAR-2025, Total Amount: 50.30, Status: Processing, Order Timestamp: 29-MAR-2025 09:12:13
Order ID: 2, Customer ID: 2, Order Date: 29-MAR-2025, Total Amount: 75.50, Status: Shipped, Order Timestamp: 29-MAR-2025 09:12:13
Order ID: 3, Customer ID: 3, Order Date: 29-MAR-2025, Total Amount: 22.90, Status: Delivered, Order Timestamp: 29-MAR-2025 09:12:13
Order ID: 4, Customer ID: 4, Order Date: 29-MAR-2025, Total Amount: 18.20, Status: Cancelled, Order Timestamp: 29-MAR-2025 09:12:13
Order ID: 5, Customer ID: 5, Order Date: 29-MAR-2025, Total Amount: 42.60, Status: Processing, Order Timestamp: 29-MAR-2025 09:12:13
Order ID: 6, Customer ID: 6, Order Date: 29-MAR-2025, Total Amount: 36.10, Status: Pending, Order Timestamp: 29-MAR-2025 09:12:13
Order ID: 7, Customer ID: 7, Order Date: 29-MAR-2025, Total Amount: 64.50, Status: Shipped, Order Timestamp: 29-MAR-2025 09:12:13
Order ID: 8, Customer ID: 8, Order Date: 29-MAR-2025, Total Amount: 88.00, Status: Delivered, Order Timestamp: 29-MAR-2025 09:12:13
Order ID: 9, Customer ID: 9, Order Date: 29-MAR-2025, Total Amount: 50.00, Status: Cancelled, Order Timestamp: 29-MAR-2025 09:12:13
Order ID: 10, Customer ID: 10, Order Date: 29-MAR-2025, Total Amount: 120.75, Status: Delivered, Order Timestamp: 29-MAR-2025 09:12:13
Order ID: 11, Customer ID: 11, Order Date: 29-MAR-2025, Total Amount: 150.20, Status: Shipped, Order Timestamp: 29-MAR-2025 09:12:13
Order ID: 12, Customer ID: 12, Order Date: 29-MAR-2025, Total Amount: 55.30, Status: Pending, Order Timestamp: 29-MAR-2025 09:12:13
Order ID: 13, Customer ID: 13, Order Date: 29-MAR-2025, Total Amount: 48.90, Status: Delivered, Order Timestamp: 29-MAR-2025 09:12:13
Order ID: 14, Customer ID: 14, Order Date: 29-MAR-2025, Total Amount: 30.60, Status: Cancelled, Order Timestamp: 29-MAR-2025 09:12:13
Order ID: 15, Customer ID: 15, Order Date: 29-MAR-2025, Total Amount: 90.50, Status: Processing, Order Timestamp: 29-MAR-2025 09:12:13
Order ID: 16, Customer ID: 16, Order Date: 29-MAR-2025, Total Amount: 18.20, Status: Pending, Order Timestamp: 29-MAR-2025 09:12:13

Figure 16-Explicit conversion using built in function(results)

- Implicit conversion to convert varchar into number dynamically

```

DECLARE
    v_order_id NUMBER := 1;           -- Example OrderID
    v_total_quantity NUMBER := 0;     -- Variable to store total quantity of products ordered
    v_total_price NUMBER(10,2) := 0;  -- Variable to store total price of products ordered
    CURSOR v_order_details IS         -- Cursor to get Order Details for a particular OrderID
        SELECT p.Name, od.Quantity, od.Price
        FROM OrderDetails od
        JOIN Products p ON od.ProductID = p.ProductID
        WHERE od.OrderID = v_order_id;
BEGIN
    -- Loop through the OrderDetails for the given OrderID
    FOR r IN v_order_details LOOP
        v_total_quantity := v_total_quantity + r.Quantity;
        v_total_price := v_total_price + (r.Quantity * r.Price);
    END LOOP;

    -- Implicit conversion: Convert the numeric total price to VARCHAR2 for concatenation with a string
    DBMS_OUTPUT.PUT_LINE('Total quantity for OrderID ' || v_order_id || ' is: ' || v_total_quantity);
    DBMS_OUTPUT.PUT_LINE('Total price for OrderID ' || v_order_id || ' is: ' || TO_CHAR(v_total_price, '9999.99'));

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('No order found with OrderID ' || v_order_id);
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END;
/

```

Figure 17-Implicit conversion to convert varchar into number dynamically



```

Script Output x
Task completed in 0.061 seconds

Total quantity for OrderID 1 is: 8
Total price for OrderID 1 is: 17.90

PL/SQL procedure successfully completed.

```

Figure 18-Implicit conversion to convert varchar into number dynamically(results)

Data Manipulation Language (DML) Commands

- Insert Data to Product table using sequence

```
CREATE SEQUENCE Products_seq START WITH 21 INCREMENT BY 1;

BEGIN
  -- Insert Sample Product 1
  INSERT INTO Products (ProductID, SupplierID, RestaurantID, Name, Description, Price, Stock, Category, ImageURL, CreatedAt)
  VALUES (Products_seq.NEXTVAL, 1, 1, 'Apple', 'Fresh Red Apples', 2.99, 100, 'Fruits', 'http://example.com/apple.jpg', CURRENT_TIMESTAMP);

  -- Insert Sample Product 2
  INSERT INTO Products (ProductID, SupplierID, RestaurantID, Name, Description, Price, Stock, Category, ImageURL, CreatedAt)
  VALUES (Products_seq.NEXTVAL, 2, 2, 'Milk', 'Organic Whole Milk', 1.49, 200, 'Dairy', 'http://example.com/milk.jpg', CURRENT_TIMESTAMP);

  -- Insert Sample Product 3
  INSERT INTO Products (ProductID, SupplierID, RestaurantID, Name, Description, Price, Stock, Category, ImageURL, CreatedAt)
  VALUES (Products_seq.NEXTVAL, 3, 3, 'Bread', 'Whole Wheat Bread', 3.59, 50, 'Baked Goods', 'http://example.com/bread.jpg', CURRENT_TIMESTAMP);

  -- Insert Sample Product 4
  INSERT INTO Products (ProductID, SupplierID, RestaurantID, Name, Description, Price, Stock, Category, ImageURL, CreatedAt)
  VALUES (Products_seq.NEXTVAL, 4, 4, 'Carrot', 'Fresh Carrots', 1.19, 150, 'Vegetables', 'http://example.com/carrot.jpg', CURRENT_TIMESTAMP);

  -- Insert Sample Product 5
  INSERT INTO Products (ProductID, SupplierID, RestaurantID, Name, Description, Price, Stock, Category, ImageURL, CreatedAt)
  VALUES (Products_seq.NEXTVAL, 5, 5, 'Handmade Vase', 'Handcrafted Ceramic Vase', 15.99, 20, 'Crafts', 'http://example.com/vase.jpg', CURRENT_TIMESTAMP);

  -- Commit the transaction
  COMMIT;
END;
/
```

Figure 19-Insert Data to Product table using sequence

- Updating data of product table

```
DECLARE
  v_product_id NUMBER := 3; -- Declare a variable for ProductID
  v_new_price NUMBER := 5.99; -- Declare a variable for the new price
  v_new_stock NUMBER := 50; -- Declare a variable for the new stock quantity
BEGIN
  -- Check condition before performing the update
  IF v_product_id IS NOT NULL THEN
    UPDATE Products
    SET
      Price = v_new_price, -- Update price
      Stock = v_new_stock -- Update stock quantity
    WHERE
      ProductID = v_product_id; -- Use the declared variable for ProductID
    COMMIT; -- Commit the transaction to make changes permanent
  ELSE
    DBMS_OUTPUT.PUT_LINE('Invalid ProductID. Update not performed.');
```

Figure 20-Updating data of product table

- Delete data from product table

```
DECLARE
  v_product_id NUMBER := 25; -- Declare a variable for ProductID
BEGIN
  -- Check condition before performing the delete
  IF v_product_id IS NOT NULL THEN
    DELETE FROM Products
    WHERE ProductID = v_product_id; -- Use the declared variable for ProductID

    COMMIT; -- Commit the transaction to make changes permanent
    DBMS_OUTPUT.PUT_LINE('Product with ProductID ' || v_product_id || ' has been deleted.');
```

Figure 21-Delete data from product table

- Group by commands to view maximum, minimum, total, average and count number of prices in each category

```

DECLARE
    v_category VARCHAR2(50);
    v_max_price NUMBER;
    v_min_price NUMBER;
    v_total_stock NUMBER;
    v_average_price NUMBER;
    v_product_count NUMBER;
BEGIN
    -- Loop over product categories (example with Fruits and Vegetables)
    FOR v_category IN (SELECT DISTINCT Category FROM Products) LOOP
        -- Find the maximum price for the current category
        SELECT MAX(Price) INTO v_max_price
        FROM Products
        WHERE Category = v_category.Category;

        -- Find the minimum price for the current category
        SELECT MIN(Price) INTO v_min_price
        FROM Products
        WHERE Category = v_category.Category;

        -- Find the total stock for the current category
        SELECT SUM(Stock) INTO v_total_stock
        FROM Products
        WHERE Category = v_category.Category;

        -- Find the average price for the current category
        SELECT AVG(Price) INTO v_average_price
        FROM Products
        WHERE Category = v_category.Category;
    END LOOP;
END;

```

Figure 23-Group by commands to view maximum, minimum, total, average and count number of prices in each category

```
-- Count the number of products in the current category
SELECT COUNT(*) INTO v_product_count
FROM Products
WHERE Category = v_category.Category;

-- Output the results
DBMS_OUTPUT.PUT_LINE('Category: ' || v_category.Category);
DBMS_OUTPUT.PUT_LINE('Max Price: ' || v_max_price);
DBMS_OUTPUT.PUT_LINE('Min Price: ' || v_min_price);
DBMS_OUTPUT.PUT_LINE('Total Stock: ' || v_total_stock);
DBMS_OUTPUT.PUT_LINE('Average Price: ' || v_average_price);
DBMS_OUTPUT.PUT_LINE('Product Count: ' || v_product_count);
DBMS_OUTPUT.PUT_LINE('-----');

END /LTP/;
```

[illegible]

Figure 22-Group by commands to view maximum, minimum, total, average and count number of prices in each category(results)

- Group by commands to find suppliers who supplied more than 3 various products using having clause

```

DECLARE
-- Declare variables for SupplierID and Product Count
v_SupplierID Products.SupplierID%TYPE;
v_ProductCount NUMBER;
BEGIN
-- Loop through each supplier's product count and display the results
FOR supplier_record IN
    (SELECT SupplierID, COUNT(ProductID) AS ProductCount
     FROM Products
     GROUP BY SupplierID
     HAVING COUNT(ProductID) > 3)
LOOP
-- Assign values to variables
v_SupplierID := supplier_record.SupplierID;
v_ProductCount := supplier_record.ProductCount;

-- Display the result using DBMS_OUTPUT
DBMS_OUTPUT.PUT_LINE('SupplierID: ' || v_SupplierID || ', Product Count: ' || v_ProductCount);
END LOOP;
END;
/

```

Script Output x

Task completed in 0.042 seconds

```

SupplierID: 1, Product Count: 6
SupplierID: 2, Product Count: 5
SupplierID: 4, Product Count: 5
SupplierID: 5, Product Count: 4
SupplierID: 3, Product Count: 5

PL/SQL procedure successfully completed.

```

Figure 24-Group by commands to find suppliers who supplied more than 3 various products using having clause

- Sub query using IN to get the suppliers who supplied product stock more than 100

```

DECLARE
CURSOR supplier_cursor IS
    SELECT p.SupplierID, SUM(p.Stock) AS TotalStock
    FROM Products p
    WHERE p.SupplierID IN
        (SELECT SupplierID
         FROM Products
         GROUP BY SupplierID
         HAVING SUM(Stock) > 100)
    GROUP BY p.SupplierID;
BEGIN
FOR supplier IN supplier_cursor LOOP
    DBMS_OUTPUT.PUT_LINE('Supplier ID: ' || supplier.SupplierID ||
                        ', Total Stock: ' || supplier.TotalStock);
END LOOP;
END;

```

Script Output x

Task completed in 0.046 seconds

```

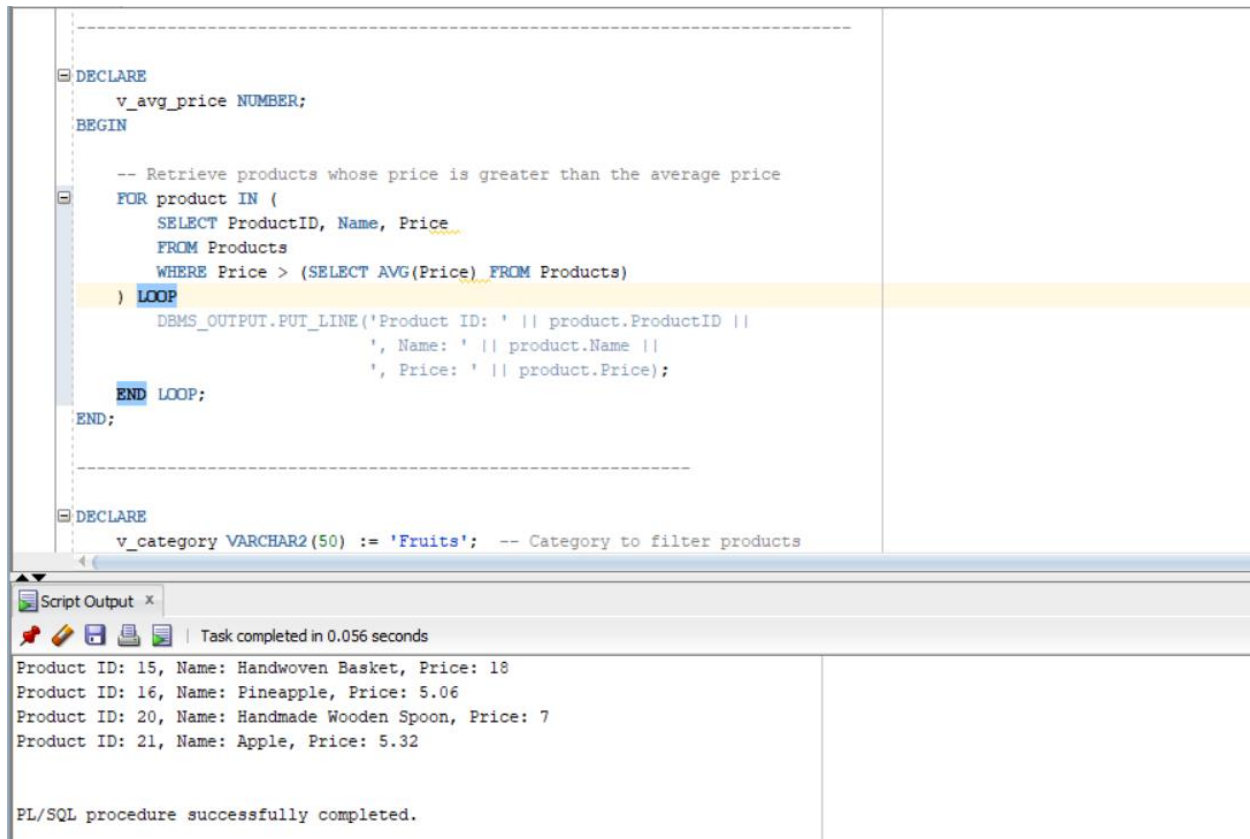
Supplier ID: 2, Total Stock: 770
Supplier ID: 4, Total Stock: 395
Supplier ID: 5, Total Stock: 130
Supplier ID: 3, Total Stock: 330

PL/SQL procedure successfully completed.

```

Figure 25-Sub query using IN to get the suppliers who supplied product stock more than 100

- Sub query using **operators** to get product details which price is higher than average product price



```
DECLARE
    v_avg_price NUMBER;
BEGIN
    -- Retrieve products whose price is greater than the average price
    FOR product IN (
        SELECT ProductID, Name, Price
        FROM Products
        WHERE Price > (SELECT AVG(Price) FROM Products)
    ) LOOP
        DBMS_OUTPUT.PUT_LINE('Product ID: ' || product.ProductID ||
                               ', Name: ' || product.Name ||
                               ', Price: ' || product.Price);
    END LOOP;
END;
```

```
DECLARE
    v_category VARCHAR2(50) := 'Fruits'; -- Category to filter products
```

Script Output x

Task completed in 0.056 seconds

Product ID: 15, Name: Handwoven Basket, Price: 18
Product ID: 16, Name: Pineapple, Price: 5.06
Product ID: 20, Name: Handmade Wooden Spoon, Price: 7
Product ID: 21, Name: Apple, Price: 5.32

PL/SQL procedure successfully completed.

Figure 26-Sub query using operators to get product details which price is higher than average product price

- Sub query using ANY and ALL to retrieve product details

```

DECLARE
v_category VARCHAR2(50) := 'Fruits'; -- Category to filter products
v_price_limit NUMBER := 2.0; -- Price limit for comparison
BEGIN
-- Example of ANY: Check if any product in the 'Fruits' category has a price greater than 50
DBMS_OUTPUT.PUT_LINE('--- Products with price > ANY of the products with price <= ' || v_price_limit || ' ---');
FOR product IN (
    SELECT ProductID, Name, Price
    FROM Products
    WHERE Category = v_category
    AND Price > ANY (SELECT Price FROM Products WHERE Price <= v_price_limit)
) LOOP
    DBMS_OUTPUT.PUT_LINE('Product ID: ' || product.ProductID ||
        ', Name: ' || product.Name ||
        ', Price: ' || product.Price);
END LOOP;

-- Separator Line
DBMS_OUTPUT.PUT_LINE('-----');

-- Example of ALL: Check if all products in the 'Fruits' category have a price greater than 50
DBMS_OUTPUT.PUT_LINE('--- Products with price > ALL of the products with price <= ' || v_price_limit || ' ---');
FOR product IN (
    SELECT ProductID, Name, Price
    FROM Products
    WHERE Category = v_category
    AND Price > ALL (SELECT Price FROM Products WHERE Price <= v_price_limit)
) LOOP
    DBMS_OUTPUT.PUT_LINE('Product ID: ' || product.ProductID ||
        ', Name: ' || product.Name ||
        ', Price: ' || product.Price);
END LOOP;
END;

```

Figure 27-Sub query using ANY and ALL to retrieve product details

```

Script Output x
Task completed in 0.048 seconds

--- Products with price > ANY of the products with price <= 2 ---
Product ID: 1, Name: Fresh Apples, Price: 3.6
Product ID: 6, Name: Juicy Oranges, Price: 3.99
Product ID: 11, Name: Bananas, Price: 1.6
Product ID: 16, Name: Pineapple, Price: 5.06
Product ID: 21, Name: Apple, Price: 5.32
-----
--- Products with price > ALL of the products with price <= 2 ---
Product ID: 1, Name: Fresh Apples, Price: 3.6
Product ID: 6, Name: Juicy Oranges, Price: 3.99
Product ID: 16, Name: Pineapple, Price: 5.06
Product ID: 21, Name: Apple, Price: 5.32

PL/SQL procedure successfully completed.

```

Figure 28-Sub query using ANY and ALL to retrieve product details

- Merge two tables Product_update and Product

```
CREATE TABLE Products_Update (
  ProductID NUMBER PRIMARY KEY,
  SupplierID NUMBER NOT NULL,
  RestaurantID NUMBER NOT NULL,
  Name VARCHAR2(255) NOT NULL,
  Description CLOB,
  Price NUMBER(10,2) NOT NULL,
  Stock NUMBER DEFAULT 0 NOT NULL,
  Category VARCHAR2(50) CHECK (Category IN ('Fruits', 'Vegetables', 'Dairy', 'Baked Goods', 'Crafts')),
  ImageURL VARCHAR2(500),
  CreatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_product_supplier_update FOREIGN KEY (SupplierID) REFERENCES Suppliers(SupplierID) ON DELETE CASCADE,
  CONSTRAINT fk_product_restaurant_update FOREIGN KEY (RestaurantID) REFERENCES Restaurants(RestaurantID) ON DELETE CASCADE
);
```

Figure 29-Merge two tables Product_update and Product

```
-- MERGE operation to update or insert data from Products_Update into Products
MERGE INTO Products_Update p_update
USING Products p
ON (p_update.ProductID = p.ProductID)
WHEN MATCHED THEN
  UPDATE SET
    p_update.SupplierID = p.SupplierID,
    p_update.RestaurantID = p.RestaurantID,
    p_update.Name = p.Name,
    p_update.Description = p.Description,
    p_update.Price = p.Price,
    p_update.Stock = p.Stock,
    p_update.Category = p.Category,
    p_update.ImageURL = p.ImageURL,
    p_update.CreatedAt = p.CreatedAt
WHEN NOT MATCHED THEN
  INSERT (ProductID, SupplierID, RestaurantID, Name, Description, Price, Stock, Category, ImageURL, CreatedAt)
  VALUES (p.ProductID, p.SupplierID, p.RestaurantID, p.Name, p.Description, p.Price, p.Stock, p.Category, p.ImageURL, p.CreatedAt);

-- Check the final data in Products table
```

Script Output x

Task completed in 0.061 seconds

25 rows merged.

Figure 30-Merge two tables Product_update and Product

JOINS AND LOOPS

- Inner join customer and order table

```
--Retrieve orders and their associated customers

SELECT o.OrderID, o.OrderDate, c.Name AS CustomerName, o.TotalAmount
FROM Orders o
INNER JOIN Customers c ON o.CustomerID = c.CustomerID;
```

Query Result x

All Rows Fetched: 20 in 0.424 seconds

ORDERID	ORDERDATE	CUSTOMERNAME	TOTALAMOUNT
1	1 29-MAR-25 09.12.13.121000000	AM Alice Johnson	50.3
2	2 29-MAR-25 09.12.13.128000000	AM Michael Smith	75.5
3	3 29-MAR-25 09.12.13.134000000	AM Emma Williams	22.9
4	4 29-MAR-25 09.12.13.140000000	AM James Brown	18.2
5	5 29-MAR-25 09.12.13.144000000	AM Sophia Davis	42.6
6	6 29-MAR-25 09.12.13.150000000	AM Daniel Miller	36.1
7	7 29-MAR-25 09.12.13.156000000	AM Olivia Garcia	64.5
8	8 29-MAR-25 09.12.13.162000000	AM William Martinez	88
9	9 29-MAR-25 09.12.13.167000000	AM Isabella Rodriguez	50
10	10 29-MAR-25 09.12.13.172000000	AM Benjamin Lee	120.75
11	11 29-MAR-25 09.12.13.178000000	AM Charlotte Walker	150.2
12	12 29-MAR-25 09.12.13.185000000	AM Ethan Hall	55.3
13	13 29-MAR-25 09.12.13.190000000	AM Mia Allen	48.9
14	14 29-MAR-25 09.12.13.196000000	AM Alexander Young	30.6
15	15 29-MAR-25 09.12.13.200000000	AM Amelia Hernandez	90.5
16	16 29-MAR-25 09.12.13.207000000	AM Lucas King	18.2
17	17 29-MAR-25 09.12.13.213000000	AM Harper Scott	72.9
18	18 29-MAR-25 09.12.13.219000000	AM Daniela Lopez	38.4
19	19 29-MAR-25 09.12.13.223000000	AM Henry Adams	11.7
20	20 29-MAR-25 09.12.13.229000000	AM Grace White	65.8

Figure 31-Inner join customer and order table

- Left Outer join with product and suppliers to view all product data with related supplier details

```
--Retrieve all products and their suppliers (even products with no supplier)

SELECT p.ProductID, p.Name AS ProductName, s.Name AS SupplierName
FROM Products p
LEFT OUTER JOIN Suppliers s ON p.SupplierID = s.SupplierID;
```

Query Result x

All Rows Fetched: 25 in 0.024 seconds

PRODUCTID	PRODUCTNAME	SUPPLIERNAME
1	1 Fresh Apples	FreshFarm Produce
2	6 Juicy Oranges	FreshFarm Produce
3	11 Bananas	FreshFarm Produce
4	16 Pineapple	FreshFarm Produce
5	21 Apple	FreshFarm Produce
6	26 Burger	FreshFarm Produce
7	2 Organic Carrots	Organic Harvest
8	7 Green Peppers	Organic Harvest
9	12 Red Tomatoes	Organic Harvest
10	17 Cabbage	Organic Harvest
11	22 Milk	Organic Harvest
12	3 Whole Milk	Local Dairy Co.
13	8 Cheddar Cheese	Local Dairy Co.
14	13 Greek Yogurt	Local Dairy Co.
15	18 Butter	Local Dairy Co.
16	23 Bread	Local Dairy Co.
17	4 Sourdough Bread	Golden Wheat Bakery
18	9 Chocolate Croissant	Golden Wheat Bakery
19	14 French Baguette	Golden Wheat Bakery
20	19 Blueberry Muffin	Golden Wheat Bakery
21	24 Carrot	Golden Wheat Bakery
22	29 Breaded Chicken	Golden Wheat Bakery

Figure 32-Left Outer join with product and suppliers to view all product data with related supplier details

- Right Outer join with suppliers and product to view all suppliers with their product details

```

--Retrieve all suppliers and the products they supply (even suppliers with no products)
SELECT s.SupplierID, s.Name AS SupplierName, p.Name AS ProductName
FROM Suppliers s
RIGHT OUTER JOIN Products p ON s.SupplierID = p.SupplierID;

```

Query Result x

All Rows Fetched: 25 in 0.01 seconds

SUPPLIERID	SUPPLIERNAME	PRODUCTNAME
1	1 FreshFarm Produce	Fresh Apples
2	1 FreshFarm Produce	Juicy Oranges
3	1 FreshFarm Produce	Bananas
4	1 FreshFarm Produce	Pineapple
5	1 FreshFarm Produce	Apple
6	1 FreshFarm Produce	Burger
7	2 Organic Harvest	Organic Carrots
8	2 Organic Harvest	Green Peppers
9	2 Organic Harvest	Red Tomatoes
10	2 Organic Harvest	Cabbage
11	2 Organic Harvest	Milk
12	3 Local Dairy Co.	Whole Milk
13	3 Local Dairy Co.	Cheddar Cheese
14	3 Local Dairy Co.	Greek Yogurt
15	3 Local Dairy Co.	Butter
16	3 Local Dairy Co.	Bread
17	4 Golden Wheat Bakery	Sourdough Bread
18	4 Golden Wheat Bakery	Chocolate Croissant
19	4 Golden Wheat Bakery	French Baguette
20	4 Golden Wheat Bakery	Blueberry Muffin

Figure 33-Right Outer join with suppliers and product to view all suppliers with their product details

- Full Outer join to view all suppliers and product details even products without suppliers and suppliers without products are included

```
--Retrieve all customers and their orders (even if a customer has no orders or an order has no associated customer)
SELECT c.Name AS CustomerName, o.OrderID, o.TotalAmount
FROM Customers c
FULL OUTER JOIN Orders o ON c.CustomerID = o.CustomerID;
```

Query Result x

Query Result 1 x

SQL | All Rows Fetched: 20 in 0.025 seconds

CUSTOMERNAME	ORDERID	TOTALAMOUNT
1 Alice Johnson	1	50.3
2 Michael Smith	2	75.5
3 Emma Williams	3	22.9
4 James Brown	4	18.2
5 Sophia Davis	5	42.6
6 Daniel Miller	6	36.1
7 Olivia Garcia	7	64.5
8 William Martinez	8	88
9 Isabella Rodriguez	9	50
10 Benjamin Lee	10	120.75
11 Charlotte Walker	11	150.2
12 Ethan Hall	12	55.3
13 Mia Allen	13	48.9
14 Alexander Young	14	30.6
15 Amelia Hernandez	15	90.5
16 Lucas King	16	18.2
17 Harper Scott	17	72.9
18 Daniela Lopez	18	38.4
19 Henry Adams	19	11.7
20 Grace White	20	65.8

Figure 34-Full Outer join to view all suppliers and product details even products without suppliers and suppliers without products are included

IF THEN ELSE EXECUTIONS

- Check order status using if condition and update to next status

```
DECLARE
    v_order_id Orders.OrderID%TYPE; -- Variable to hold OrderID
    v_status Orders.Status%TYPE;    -- Variable to hold Order Status
BEGIN
    -- Example: Set a specific order ID to check
    v_order_id := 10;

    -- Retrieve the status of the order
    SELECT Status
    INTO v_status
    FROM Orders
    WHERE OrderID = v_order_id;

    -- Conditional logic based on Order Status
    IF v_status = 'Pending' THEN
        -- Update order status to 'Processing'
        UPDATE Orders
        SET Status = 'Processing'
        WHERE OrderID = v_order_id;

        DBMS_OUTPUT.PUT_LINE('Order status changed to Processing.');
```

Figure 35-Check order status using if condition and update to next status-

```
ELSIF v_status = 'Shipped' THEN
    -- Update order status to 'Delivered'
    UPDATE Orders
    SET Status = 'Delivered'
    WHERE OrderID = v_order_id;

    DBMS_OUTPUT.PUT_LINE('Order status changed to Delivered.');
```

```
ELSE
    -- If the order is already Delivered or Cancelled, no update
    DBMS_OUTPUT.PUT_LINE('No further action required, order is either Delivered or Cancelled.');
```

```
END IF;
```

```
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Order not found with the provided OrderID.');
```

```
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('An error occurred: ' || SQLERRM);
```

```
END;
```

```
/
```

Script Output x

Task completed in 0.114 seconds

Order status changed to Processing.

PL/SQL procedure successfully completed.

Figure 36-Check order status using if condition and update to next status

LOOPS IN PL SQL

- Using a loop to fetch product tables multiple rows

```
DECLARE
    CURSOR product_cursor IS
        SELECT ProductID, Name, Price, Category
        FROM Products;

    v_product_id Products.ProductID%TYPE;
    v_name Products.Name%TYPE;
    v_price Products.Price%TYPE;
    v_category Products.Category%TYPE;
BEGIN
    -- Open the cursor
    OPEN product_cursor;

    -- Start the loop
    LOOP
        -- Fetch a row from the cursor
        FETCH product_cursor INTO v_product_id, v_name, v_price, v_category;

        -- Exit the loop when no more rows are found
        EXIT WHEN product_cursor%NOTFOUND;

        -- Print the product details
        DBMS_OUTPUT.PUT_LINE('Product ID: ' || v_product_id);
        DBMS_OUTPUT.PUT_LINE('Name: ' || v_name);
        DBMS_OUTPUT.PUT_LINE('Price: ' || v_price);
        DBMS_OUTPUT.PUT_LINE('Category: ' || v_category);
        DBMS_OUTPUT.PUT_LINE('-----');
    END LOOP;

    -- Close the cursor
    CLOSE product_cursor;
END;
```

Figure 37-Using a loop to fetch product tables multiple rows

```
EXCEPTION
    WHEN OTHERS THEN
        -- Handle the error
        IF product_cursor%ISOPEN THEN
            CLOSE product_cursor; -- Ensure cursor is closed on error
        END IF;
        DBMS_OUTPUT.PUT_LINE('An error occurred: ' || SQLERRM);
END;
```

Script Output x

Task completed in 0.068 seconds

Product ID: 26
Name: Burger
Price: 2.99
Category: Baked Goods

PL/SQL procedure successfully completed.

Figure 38-Using a loop to fetch product tables multiple rows

- Using while Loop to fetch product tables details

```

DECLARE
    CURSOR product_cursor IS
        SELECT ProductID, Name, Price, Category, Stock
        FROM Products;

    v_product_id Products.ProductID%TYPE;
    v_name Products.Name%TYPE;
    v_price Products.Price%TYPE;
    v_category Products.Category%TYPE;
    v_stock Products.Stock%TYPE;
    v_counter INTEGER := 0;
BEGIN
    -- Open the cursor
    OPEN product_cursor;

    -- Fetch the first row
    FETCH product_cursor INTO v_product_id, v_name, v_price, v_category, v_stock;

    -- While loop to process all rows
    WHILE product_cursor%FOUND LOOP
        -- Print product details
        DBMS_OUTPUT.PUT_LINE('Product ID: ' || v_product_id);
        DBMS_OUTPUT.PUT_LINE('Name: ' || v_name);
        DBMS_OUTPUT.PUT_LINE('Price: ' || v_price);
        DBMS_OUTPUT.PUT_LINE('Category: ' || v_category);
        DBMS_OUTPUT.PUT_LINE('Stock: ' || v_stock);
        DBMS_OUTPUT.PUT_LINE('-----');

        -- Fetch the next row
        FETCH product_cursor INTO v_product_id, v_name, v_price, v_category, v_stock;
    END LOOP;

    -- Close the cursor after processing
    CLOSE product_cursor;

```

Figure 39-Using while Loop to fetch product tables details

```

EXCEPTION
    WHEN OTHERS THEN
        -- Handle any exceptions and ensure cursor is closed
        IF product_cursor%ISOPEN THEN
            CLOSE product_cursor;
        END IF;
        DBMS_OUTPUT.PUT_LINE('An error occurred: ' || SQLERRM);
END;

```

Script Output x

Task completed in 0.116 seconds

```

Product ID: 1
Name: Fresh Apples
Price: 3.6
Category: Fruits
Stock: 110
-----
Product ID: 2
Name: Organic Carrots
Price: 2
Category: Vegetables
Stock: 210
-----
Product ID: 3
Name: Whole Milk
Price: 5.99
Category: Dairy
Stock: 50
-----
Product ID: 4
Name: Sourdough Bread
Price: 4.5

```

Figure 40-Using while Loop to fetch product tables details

- Using for loop to retrieve product table details

```

BEGIN
  -- FOR loop to iterate through each product
  FOR product_record IN (SELECT ProductID, Name, Price, Category, Stock FROM Products) LOOP
    -- Print product details
    DBMS_OUTPUT.PUT_LINE('Product ID: ' || product_record.ProductID);
    DBMS_OUTPUT.PUT_LINE('Name: ' || product_record.Name);
    DBMS_OUTPUT.PUT_LINE('Price: ' || product_record.Price);
    DBMS_OUTPUT.PUT_LINE('Category: ' || product_record.Category);
    DBMS_OUTPUT.PUT_LINE('Stock: ' || product_record.Stock);
    DBMS_OUTPUT.PUT_LINE('-----');
  END LOOP;
END;
/

```

Script Output x

Task completed in 0.124 seconds

```

-----
Product ID: 24
Name: Carrot
Price: 1.19
Category: Vegetables
Stock: 150
-----
Product ID: 26
Name: Burger
Price: 2.99
Category: Baked Goods
Stock: 150
-----

PL/SQL procedure successfully completed.

```

Figure 41-Using for loop to retrieve product table details

- Using GOTO label to find expensive product which is higher than 10.0

```

DECLARE
  CURSOR product_cursor IS
    SELECT ProductID, Name, Price, Category
    FROM Products;

  -- Declare variables to store the fetched product details
  v_product Products.ProductID%TYPE;
  v_name Products.Name%TYPE;
  v_price Products.Price%TYPE;
  v_category Products.Category%TYPE;
BEGIN
  -- Open the cursor
  OPEN product_cursor;

  -- Loop through the products
  LOOP
    -- Fetch the current product data into variables
    FETCH product_cursor INTO v_product, v_name, v_price, v_category;

    -- Exit the loop when no more products are found
    EXIT WHEN product_cursor%NOTFOUND;

    -- If the price is greater than 10, jump to the label
    IF v_price > 10 THEN
      GOTO expensive_product;
    END IF;
  END LOOP;

  -- If no expensive product was found, this part will be executed
  DBMS_OUTPUT.PUT_LINE('No product with a price greater than 10 found.');
```

Figure 42-Using GOTO label to find expensive product which is higher than 10.0

```

  -- Label for expensive products
  <<expensive_product>>
  DBMS_OUTPUT.PUT_LINE('Expensive product found: ' || v_name || ' with price: ' || v_price);

  -- Close the cursor
  CLOSE product_cursor;
END;
/

```

Script Output x

Task completed in 0.067 seconds

```

Expensive product found: Handmade Pottery Vase with price: 15

PL/SQL procedure successfully completed.

```

Figure 43-Using GOTO label to find expensive product which is higher than 10.0

USING EXPLICIT CURSORS

- Using cursor to fetch a single row from product table

```
DECLARE
-- Declare the cursor to select product details from the Products table
CURSOR product_cursor IS
    SELECT ProductID, Name, Price, Category
    FROM Products where ProductID=3;

-- Declare variables to store the fetched values
v_productID Products.ProductID%TYPE;
v_name Products.Name%TYPE;
v_price Products.Price%TYPE;
v_category Products.Category%TYPE;
BEGIN
-- Open the cursor
OPEN product_cursor;

-- Fetch the first record into variables
FETCH product_cursor INTO v_productID, v_name, v_price, v_category;

-- Check if the fetch was successful
IF product_cursor%FOUND THEN
    -- Output the fetched product details
    DBMS_OUTPUT.PUT_LINE('Product ID: ' || v_productID);
    DBMS_OUTPUT.PUT_LINE('Product Name: ' || v_name);
    DBMS_OUTPUT.PUT_LINE('Product Price: ' || v_price);
    DBMS_OUTPUT.PUT_LINE('Product Category: ' || v_category);
ELSE
    -- If no rows are found
    DBMS_OUTPUT.PUT_LINE('No products found.');
```

Figure 44-Using cursor to fetch a single row from product table

```
Script Output x
Task completed in 0.083 seconds

Product ID: 3
Product Name: Whole Milk
Product Price: 5.99
Product Category: Dairy

PL/SQL procedure successfully completed.
```

Figure 45-Using cursor to fetch a single row from product table(results)

- Using cursor to fetch multiples row from product table

```
DECLARE
-- Declare the cursor to select product details from the Products table
CURSOR product_cursor IS
    SELECT ProductID, Name, Price, Category
    FROM Products;

-- Declare variables to store the fetched values
v_productID Products.ProductID%TYPE;
v_name Products.Name%TYPE;
v_price Products.Price%TYPE;
v_category Products.Category%TYPE;
BEGIN
-- Open the cursor
OPEN product_cursor;

-- Loop through each row in the cursor
LOOP
    -- Fetch the next record into variables
    FETCH product_cursor INTO v_productID, v_name, v_price, v_category;

    -- Exit the loop if no more rows are found
    EXIT WHEN product_cursor%NOTFOUND;

    -- Output the fetched product details
    DBMS_OUTPUT.PUT_LINE('Product ID: ' || v_productID);
    DBMS_OUTPUT.PUT_LINE('Product Name: ' || v_name);
    DBMS_OUTPUT.PUT_LINE('Product Price: ' || v_price);
    DBMS_OUTPUT.PUT_LINE('Product Category: ' || v_category);
    DBMS_OUTPUT.PUT_LINE('-----');
END LOOP;

-- Close the cursor
CLOSE product_cursor;
END;
```

Figure 46-Using cursor to fetch multiples row from product table

```

Product ID: 1
Product Name: Fresh Apples
Product Price: 3.6
Product Category: Fruits
-----
Product ID: 2
Product Name: Organic Carrots
Product Price: 2
Product Category: Vegetables
-----
Product ID: 3
Product Name: Whole Milk
Product Price: 5.99
Product Category: Dairy
-----
Product ID: 4
Product Name: Sourdough Bread
Product Price: 4.5
Product Category: Baked Goods
-----
Product ID: 5
Product Name: Handmade Pottery Vase
Product Price: 15
Product Category: Crafts
-----

```

Figure 47-Using cursor to fetch multiples row from product table(results)

- Parameterized cursor fetch products total price from different categories

```

-- parameterized cursor to retrieve total for each category
DECLARE
-- Define a cursor with an IN parameter
CURSOR product_cursor(p_category IN VARCHAR2) IS
    SELECT Price
    FROM Products
    WHERE Category = p_category;

-- Declare a variable to hold the total price sum
v_total_price NUMBER := 0;

-- Declare a variable to store fetched product price
v_price Products.Price%TYPE;
BEGIN
-- Open the cursor with a specific category
OPEN product_cursor('Fruits'); -- You can change 'Fruits' to any category

-- Loop through the cursor and accumulate the total price
LOOP
-- Fetch the price of each product into v_price
FETCH product_cursor INTO v_price;

-- Exit the loop when no more rows are found
EXIT WHEN product_cursor%NOTFOUND;

-- Accumulate the total price
v_total_price := v_total_price + v_price;
END LOOP;

-- Output the total price of all products in the given category
DBMS_OUTPUT.PUT_LINE('Total price of products in category "Fruits": ' || v_total_price);

-- Close the cursor
CLOSE product_cursor;
END;

```

Figure 48-Parameterized cursor fetch products total price from different categories

```

Total price of products in category "Fruits": 19.57

PL/SQL procedure successfully completed.

```

Figure 49-Parameterized cursor fetch products total price from different categories(results)

- Cursor with FOR UPDATE to update to increase product price by 10 percent

```

DECLARE
-- Define a cursor to fetch product details
CURSOR product_cursor IS
SELECT ProductID, Price
FROM Products
WHERE Category = 'Fruits' FOR UPDATE; -- You can change the category as needed

-- Variables to hold fetched data
v_product_id Products.ProductID%TYPE;
v_price Products.Price%TYPE;
BEGIN
-- Open the cursor
OPEN product_cursor;

-- Loop through the cursor to process each product
LOOP
-- Fetch the product details into variables
FETCH product_cursor INTO v_product_id, v_price;

-- Exit the loop when no more rows are found
EXIT WHEN product_cursor%NOTFOUND;

-- Update the price of the current product (example: increasing by 10%)
v_price := v_price * 1.10; -- Increase price by 10%

-- Update the row where the cursor is currently positioned
UPDATE Products
SET Price = v_price
WHERE CURRENT OF product_cursor; -- Use WHERE CURRENT OF to update the current row

END LOOP;

-- Close the cursor
CLOSE product_cursor;

```

Figure 50-Cursor with FOR UPDATE to update to increase product price by 10 percent

```

-- Close the cursor
CLOSE product_cursor;

-- Output completion message
DBMS_OUTPUT.PUT_LINE('Product prices updated successfully.');
```

END;

Script Output x

Task completed in 0.097 seconds

Product prices updated successfully.

PL/SQL procedure successfully completed.

Figure 51-Cursor with FOR UPDATE to update to increase product price by 10 percent(results)

- Cursor with sub query.

```

DECLARE
-- Define a cursor with a subquery to fetch products with price higher than the average price for a category
CURSOR product_cursor IS
SELECT ProductID, Name, Price
FROM Products
WHERE Price > (
SELECT AVG(Price)
FROM Products
WHERE Category = 'Fruits'
)
AND Category = 'Fruits';

-- Variables to hold the fetched product details
v_product_id Products.ProductID%TYPE;
v_name Products.Name%TYPE;
v_price Products.Price%TYPE;
BEGIN
-- Open the cursor
OPEN product_cursor;

-- Loop through the cursor to fetch each product
LOOP
-- Fetch the product details into variables
FETCH product_cursor INTO v_product_id, v_name, v_price;

-- Exit the loop when no more rows are found
EXIT WHEN product_cursor%NOTFOUND;

-- Output the product details
DBMS_OUTPUT.PUT_LINE('Product ID: ' || v_product_id ||
', Name: ' || v_name ||
', Price: ' || v_price);

END LOOP;

-- Close the cursor

```

Figure 52-Cursor with sub query.

```
END LOOP;

-- Close the cursor
CLOSE product_cursor;
END;
/
```

Script Output x

Task completed in 0.074 seconds

Product ID: 6, Name: Juicy Oranges, Price: 4.39
Product ID: 16, Name: Pineapple, Price: 5.57
Product ID: 21, Name: Apple, Price: 5.85

PL/SQL procedure successfully completed.

Figure 53-Cursor with sub query.(results)

EXCEPTIONS

- Pre-defined oracle server errors.

```
DECLARE
-- Declare a variable to store the product price
v_price Products.Price%TYPE;

-- Declare a variable to store the category of the product
v_category VARCHAR2(50) := 'Fruitsnew';

-- Declare a cursor to fetch product prices for the specified category
CURSOR product_cursor IS
    SELECT Price
    FROM Products
    WHERE Category = v_category;
BEGIN
-- Try to fetch the price of the first product in the 'Fruits' category
OPEN product_cursor;
FETCH product_cursor INTO v_price;

-- Handle the case when no data is found
IF product_cursor%NOTFOUND THEN
    RAISE NO_DATA_FOUND;
END IF;

-- Process the data (e.g., display product price)
DBMS_OUTPUT.PUT_LINE('Product Price: ' || v_price);

-- Close the cursor
CLOSE product_cursor;

EXCEPTION
-- Handle NO_DATA_FOUND exception
WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('No products found in the ' || v_category || ' category.');
```

Figure 54-Pre-defined oracle server errors.

```
-- Handle TOO_MANY_ROWS exception (if the cursor returns more than one row)
WHEN TOO_MANY_ROWS THEN
    DBMS_OUTPUT.PUT_LINE('Multiple products found in the ' || v_category || ' category. Please refine your query.');
```

```
-- Handle generic exceptions
WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('An unexpected error occurred: ' || SQLERRM);
END;
```

Script Output x

Task completed in 0.114 seconds

No products found in the Fruitsnew category.

PL/SQL procedure successfully completed.

Figure 55-Pre-defined oracle server errors.(results)

- too many rows exception for fetching multiple row values to single variable

```
DECLARE
v_price Products.Price%TYPE;
BEGIN
-- This will raise TOO_MANY_ROWS if there are multiple rows returned
SELECT Price
INTO v_price
FROM Products
WHERE Category = 'Fruits'; -- Multiple rows returned for the 'Fruits' category

DBMS_OUTPUT.PUT_LINE('Product Price: ' || v_price);
EXCEPTION
WHEN TOO_MANY_ROWS THEN
    DBMS_OUTPUT.PUT_LINE('More than one row returned.');
```

Script Output x

Task completed in 0.121 seconds

More than one row returned.

PL/SQL procedure successfully completed.

Figure 56-too many rows exception for fetching multiple row values to single variable

- Non-Pre-defined Exception

```

DECLARE
    v_price Products.Price%TYPE;
BEGIN
    -- This will raise TOO_MANY_ROWS if there are multiple rows returned
    SELECT Price
    INTO v_price
    FROM Products
    WHERE Category = 'Fruits'; -- Multiple rows returned for the 'Fruits' category

    DBMS_OUTPUT.PUT_LINE('Product Price: ' || v_price);
EXCEPTION
    WHEN TOO_MANY_ROWS THEN
        DBMS_OUTPUT.PUT_LINE('More than one row returned.');
```

Script Output x

Task completed in 0.121 seconds

More than one row returned.

PL/SQL procedure successfully completed.

Figure 57-Non-Pre-defined Exception

- User defined exception to avoid inserting negative values for price

```

DECLARE
    -- Declare a user-defined exception
    price_too_low EXCEPTION;

    -- Declare a variable for the product price
    v_productID NUMBER := 101;
    v_supplierID NUMBER := 10;
    v_name VARCHAR2(255) := 'Apple';
    v_description CLOB := 'Fresh apples from the farm';
    v_price NUMBER := -5.99; -- Invalid price (negative value)
    v_stock NUMBER := 50;
    v_category VARCHAR2(50) := 'Fruits';
    v_imageURL VARCHAR2(500) := 'http://example.com/apple.jpg';
BEGIN
    -- Check if the price is less than or equal to zero
    IF v_price <= 0 THEN
        -- Raise the user-defined exception if the price is invalid
        RAISE price_too_low;
    END IF;

    -- Insert data into the Products table if the price is valid
    INSERT INTO Products (ProductID, SupplierID, Name, Description, Price, Stock, Category, ImageURL)
    VALUES (v_productID, v_supplierID, v_name, v_description, v_price, v_stock, v_category, v_imageURL);

    DBMS_OUTPUT.PUT_LINE('Product added successfully!');
```

```

EXCEPTION
    -- Handle the user-defined exception
    WHEN price_too_low THEN
        DBMS_OUTPUT.PUT_LINE('Error: The product price cannot be zero or negative.');
```

```

    -- Handle any other unforeseen exceptions
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('An unexpected error occurred: ' || SQLERRM);
```

Figure 58-User defined exception to avoid inserting negative values for price

Script Output x

Task completed in 0.062 seconds

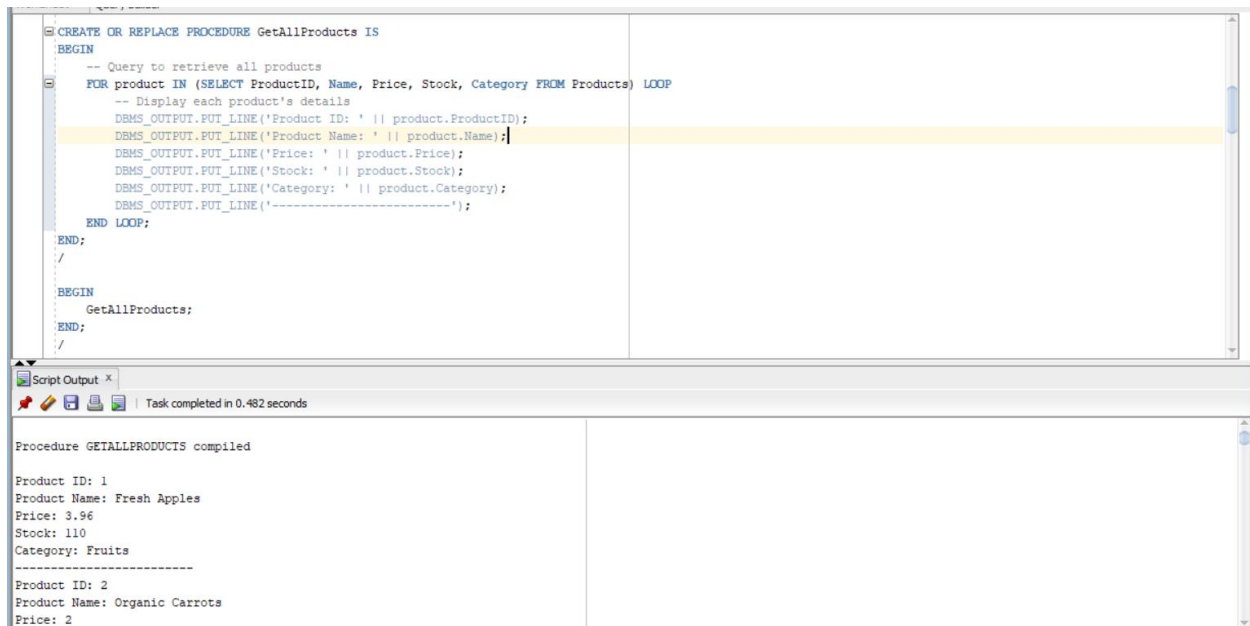
Error: The product price cannot be zero or negative.

PL/SQL procedure successfully completed.

Figure 59-User defined exception to avoid inserting negative values for price(results)

PROCEDURES

- Procedure to retrieve all products



```
CREATE OR REPLACE PROCEDURE GetAllProducts IS
BEGIN
    -- Query to retrieve all products
    FOR product IN (SELECT ProductID, Name, Price, Stock, Category FROM Products) LOOP
        -- Display each product's details
        DBMS_OUTPUT.PUT_LINE('Product ID: ' || product.ProductID);
        DBMS_OUTPUT.PUT_LINE('Product Name: ' || product.Name);
        DBMS_OUTPUT.PUT_LINE('Price: ' || product.Price);
        DBMS_OUTPUT.PUT_LINE('Stock: ' || product.Stock);
        DBMS_OUTPUT.PUT_LINE('Category: ' || product.Category);
        DBMS_OUTPUT.PUT_LINE('-----');
    END LOOP;
END;
/

BEGIN
    GetAllProducts;
END;
/
```

Script Output x

Task completed in 0.482 seconds

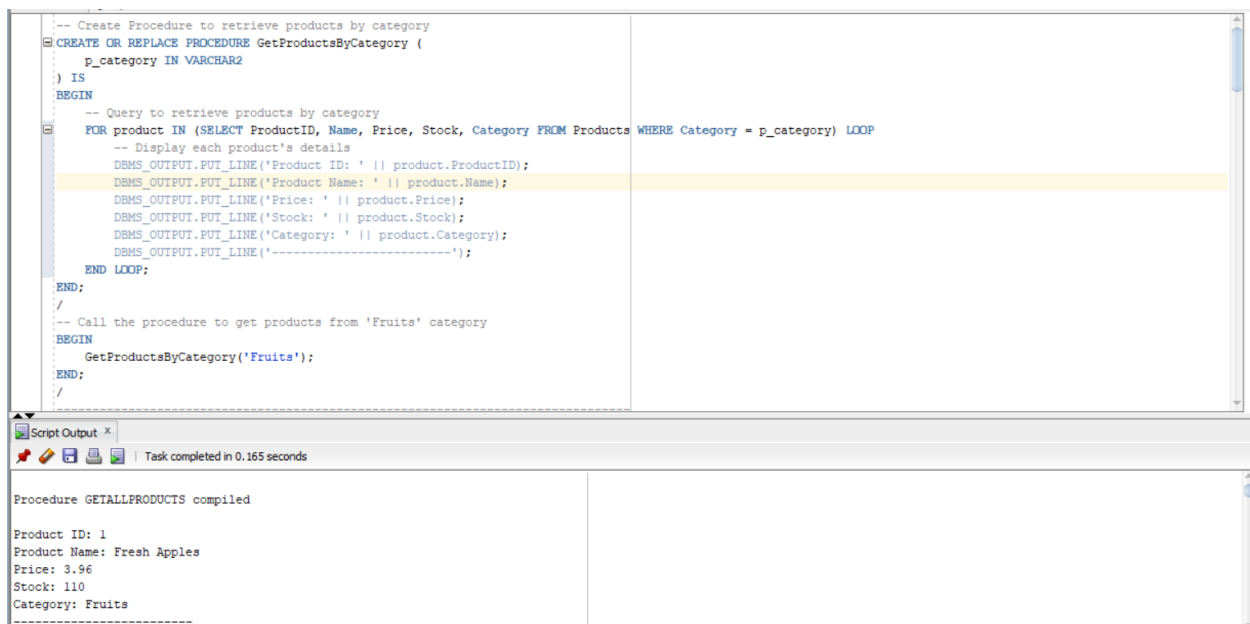
Procedure GETALLPRODUCTS compiled

Product ID: 1
Product Name: Fresh Apples
Price: 3.96
Stock: 110
Category: Fruits

Product ID: 2
Product Name: Organic Carrots
Price: 2

Figure 60-Procedure to retrieve all products

- Parameterized procedure to retrieve products by category



```
-- Create Procedure to retrieve products by category
CREATE OR REPLACE PROCEDURE GetProductsByCategory (
    p_category IN VARCHAR2
) IS
BEGIN
    -- Query to retrieve products by category
    FOR product IN (SELECT ProductID, Name, Price, Stock, Category FROM Products WHERE Category = p_category) LOOP
        -- Display each product's details
        DBMS_OUTPUT.PUT_LINE('Product ID: ' || product.ProductID);
        DBMS_OUTPUT.PUT_LINE('Product Name: ' || product.Name);
        DBMS_OUTPUT.PUT_LINE('Price: ' || product.Price);
        DBMS_OUTPUT.PUT_LINE('Stock: ' || product.Stock);
        DBMS_OUTPUT.PUT_LINE('Category: ' || product.Category);
        DBMS_OUTPUT.PUT_LINE('-----');
    END LOOP;
END;
/

-- Call the procedure to get products from 'Fruits' category
BEGIN
    GetProductsByCategory('Fruits');
END;
/
```

Script Output x

Task completed in 0.165 seconds

Procedure GETALLPRODUCTS compiled

Product ID: 1
Product Name: Fresh Apples
Price: 3.96
Stock: 110
Category: Fruits

Figure 61-Parameterized procedure to retrieve products by category

- Use IN and OUT parameters in a procedure to retrieve product stock for each category

```

CREATE OR REPLACE PROCEDURE GetProductsByCategoryWithStock (
    p_category IN VARCHAR2,          -- IN parameter: Category of the products
    p_total_stock OUT NUMBER         -- OUT parameter: Total stock of products in the category
) IS
BEGIN
    -- Initialize the total stock to 0
    p_total_stock := 0;

    -- Loop through the products matching the category
    FOR product IN (SELECT ProductID, Name, Stock FROM Products WHERE Category = p_category) LOOP
        -- Add the stock of each product to the total stock
        p_total_stock := p_total_stock + product.Stock;

        -- Display each product's details
        DBMS_OUTPUT.PUT_LINE('Product ID: ' || product.ProductID);
        DBMS_OUTPUT.PUT_LINE('Product Name: ' || product.Name);
        DBMS_OUTPUT.PUT_LINE('Stock: ' || product.Stock);
        DBMS_OUTPUT.PUT_LINE('-----');
    END LOOP;

    -- Display the total stock
    DBMS_OUTPUT.PUT_LINE('Total Stock in Category "' || p_category || '" : ' || p_total_stock);
END;
/

DECLARE
    v_total_stock NUMBER; -- Variable to store the output of the procedure
BEGIN
    -- Call the procedure and pass the category "Fruits" as the input
    GetProductsByCategoryWithStock('Fruits', v_total_stock);

    -- Display the total stock outside the procedure
    DBMS_OUTPUT.PUT_LINE('Total Stock for Fruits: ' || v_total_stock);
END;

```

Figure 62-Use IN and OUT parameters in a procedure to retrieve product stock for each category

```

Script Output x
Task completed in 0.078 seconds

Product ID: 1
Product Name: Fresh Apples
Stock: 110
-----
Product ID: 6
Product Name: Juicy Oranges
Stock: 120
-----
Product ID: 11
Product Name: Bananas
Stock: 150
-----
Product ID: 16
Product Name: Pineapple
Stock: 110
-----
Product ID: 21
Product Name: Watermelon
Stock: 130
-----
Total Stock for Fruits: 510

```

Figure 63-Use IN and OUT parameters in a procedure to retrieve product stock for each category(results)

- Function to return number of customers.

```

CREATE OR REPLACE FUNCTION GetNumberOfCustomers
RETURN NUMBER IS
    v_customer_count NUMBER;
BEGIN
    -- Count the number of customers in the Customers table
    SELECT COUNT(*) INTO v_customer_count
    FROM Customers;

    -- Return the number of customers
    RETURN v_customer_count;
END;
/

DECLARE
    v_num_customers NUMBER;
BEGIN
    v_num_customers := GetNumberOfCustomers; -- Get the number of customers
    DBMS_OUTPUT.PUT_LINE('Total number of customers: ' || v_num_customers);
END;
/

```

Figure 64-Function to return number of customers

Script Output x

Task completed in 0.1 seconds

Total number of customers: 20

PL/SQL procedure successfully completed.

Figure 65-Function to return number of customers(results)

- Parameterized function to get stock count by product id

```

CREATE OR REPLACE FUNCTION GetTotalStock (p_product_id IN NUMBER)
RETURN NUMBER IS
    v_total_stock NUMBER;
BEGIN
    SELECT Stock INTO v_total_stock
    FROM Products
    WHERE ProductID = p_product_id;

    RETURN v_total_stock;
END;
/

DECLARE
    v_stock NUMBER;
BEGIN
    v_stock := GetTotalStock(1); -- Get total stock for product with ProductID = 1
    DBMS_OUTPUT.PUT_LINE('Total stock for Product 1: ' || v_stock);
END;
/

```

Script Output x

Task completed in 0.135 seconds

Function GETTOTALSTOCK compiled

Total stock for Product 1: 110

PL/SQL procedure successfully completed.

Figure 66-Parameterized function to get stock count by product id

DATA DEFINITION LANGUAGE

- Create users

```
CREATE USER lasindu IDENTIFIED BY lasindu;  
  
CREATE USER shashimalnew IDENTIFIED BY shashimal_password;
```

Figure 67-Create users

- Grant privileges

```
GRANT CREATE SESSION TO shashimalnew;  
  
GRANT ALL PRIVILEGES TO LASINDU;
```

Figure 68-Grant privileges

- Create table shashimalnew schema

```
-- Connect as shashimalnew user  
-- Create a table inside the schema  
CREATE TABLE shashimalnew.my_table (  
    id NUMBER PRIMARY KEY,  
    name VARCHAR2(50)  
);
```

Figure 69-Create table shashimalnew schema

- Grant shashimalnew user my table privileges to lasindu user

```
-- Grant UPDATE and DELETE privileges to user LASINDU on a specific table  
GRANT UPDATE, DELETE ON shashimalnew.my_table TO LASINDU;
```

Figure 70-Grant shashimalnew user my table privileges to lasindu user

- Grant specific column privileges to lasindu user

```
GRANT UPDATE (emp_salary) ON shashimalnew.employee_info TO LASINDU;
```

Figure 71-Grant specific column privileges to lasindu user

- Create User Roles

```
CREATE ROLE COADMIN;
```

Figure 72-Create User Roles

- Grant create table Privileges to COADMIN Role with admin option

```
GRANT CREATE TABLE TO COADMIN WITH ADMIN OPTION;
```

Figure 73-Grant create table Privileges to COADMIN Role with admin option

- Grant select privilege of mytable in shashimalnew schema to COADMIN role

```
GRANT SELECT ON shashimalnew.my_table TO COADMIN;
```

Figure 74-Grant select privilege of mytable in shashimalnew schema to COADMIN role

- Assign user manager role to lasindu user

```
CREATE ROLE USERMANAGER;
```

```
GRANT USERMANAGER TO lasindu;
```

Figure 75-Assign user manager role to lasindu user

- Assign User manager role to coadmin role

```
GRANT USERMANAGER TO COADMIN;
```

Figure 76-Assign User manager role to coadmin role

NoSQL (MongoDB)

Insert Suppliers

```
db.Suppliers.insertMany([  
  { _id: 1, Name: "Supplier A", Email: "supplierA@example.com", Phone: "1234567890",  
    Address: "123 Street", JoinDate: new Date() },  
  { _id: 2, Name: "Supplier B", Email: "supplierB@example.com", Phone: "9876543210",  
    Address: "456 Avenue", JoinDate: new Date() }  
]);
```

Insert Customers

```
db.Customers.insertMany([  
  { _id: 1, Name: "John Doe", Email: "john@example.com", Phone: "9876543210", Address:  
    "789 Street", RegisterDate: new Date() },  
  { _id: 2, Name: "Jane Smith", Email: "jane@example.com", Phone: "8765432109", Address:  
    "456 Avenue", RegisterDate: new Date() }  
]);
```

Insert Restaurants

```
db.Restaurants.insertMany([  
  { _id: 1, Name: "Restaurant X", Email: "restaurantX@example.com", Phone: "1122334455",  
    Address: "12 Main St", RegisterDate: new Date() },  
  { _id: 2, Name: "Restaurant Y", Email: "restaurantY@example.com", Phone: "5566778899",  
    Address: "98 Market St", RegisterDate: new Date() }  
]);
```

Insert Products

```
db.Products.insertMany([  
  
  { _id: 1, SupplierID: 1, Name: "Apple", Description: "Fresh apples", Price: 2.99, Stock: 100,  
    Category: "Fruits", ImageURL: "apple.jpg", CreatedAt: new Date() },  
  
  { _id: 2, RestaurantID: 1, Name: "Pizza", Description: "Delicious pizza", Price: 9.99, Stock:  
    50, Category: "Baked Goods", ImageURL: "pizza.jpg", CreatedAt: new Date() }  
  
]);
```

Insert Orders

```
db.Orders.insertMany([  
  
  { _id: 1, CustomerID: 1, OrderDate: new Date(), Status: "Pending", TotalAmount: 12.98 },  
  
  { _id: 2, CustomerID: 2, OrderDate: new Date(), Status: "Shipped", TotalAmount: 9.99 }  
  
]);
```

Insert OrderDetails

```
db.OrderDetails.insertMany([  
  
  { _id: 1, OrderID: 1, ProductID: 1, Quantity: 2, Price: 5.98 },  
  
  { _id: 2, OrderID: 2, ProductID: 2, Quantity: 1, Price: 9.99 }  
  
]);
```

Insert Payments

```
db.Payments.insertMany([  
  
  { _id: 1, OrderID: 1, PaymentDate: new Date(), Amount: 12.98, PaymentMethod: "Credit  
    Card", Status: "Completed" },  
  
  { _id: 2, OrderID: 2, PaymentDate: new Date(), Amount: 9.99, PaymentMethod: "PayPal",  
    Status: "Pending" }  
  
]);
```

Insert Deliveries

```
db.Deliveries.insertMany([  
  
  { _id: 1, OrderID: 1, DeliveryAddress: "789 Street", DeliveryStatus: "Out for Delivery",  
    EstimatedDeliveryDate: new Date(), DeliveredDate: null },  
  
  { _id: 2, OrderID: 2, DeliveryAddress: "456 Avenue", DeliveryStatus: "Delivered",  
    EstimatedDeliveryDate: new Date(), DeliveredDate: new Date() }  
  
]);
```

Insert Reviews

```
db.Reviews.insertMany([  
  
  { _id: 1, CustomerID: 1, ProductID: 1, Rating: 5, ReviewText: "Excellent quality!",  
    ReviewDate: new Date() },  
  
  { _id: 2, CustomerID: 2, ProductID: 2, Rating: 4, ReviewText: "Tasty and fresh!",  
    ReviewDate: new Date() }  
  
]);
```

Insert Feedbacks

```
db.Feedbacks.insertMany([  
  
  { _id: 1, OrderID: 1, CustomerID: 1, FeedbackText: "Great service!", FeedbackDate: new  
    Date() },  
  
  { _id: 2, OrderID: 2, CustomerID: 2, FeedbackText: "Fast delivery!", FeedbackDate: new  
    Date() }  
  
]);
```

Insert many for reviews

```
db.Reviews.insertMany([  
  
  { _id: 3, CustomerID: 1, ProductID: 2, Rating: 4, ReviewText: "Great quality, very fresh!",  
    ReviewDate: new Date() },  
  
  { _id: 4, CustomerID: 2, ProductID: 3, Rating: 5, ReviewText: "Excellent product, will buy  
    again.", ReviewDate: new Date() },  
  
]);
```



```

    { _id: 5, CustomerID: 3, ProductID: 1, Rating: 3, ReviewText: "Good, but could be improved.", ReviewDate: new Date() },

    { _id: 6, CustomerID: 4, ProductID: 4, Rating: 4, ReviewText: "Loved it, would recommend.", ReviewDate: new Date() },

    { _id: 7, CustomerID: 5, ProductID: 5, Rating: 2, ReviewText: "Not as expected, quality could be better.", ReviewDate: new Date() },

    { _id: 8, CustomerID: 6, ProductID: 2, Rating: 5, ReviewText: "Amazing taste, would definitely purchase again.", ReviewDate: new Date() },

    { _id: 9, CustomerID: 7, ProductID: 3, Rating: 4, ReviewText: "Really nice, though a bit overpriced.", ReviewDate: new Date() },

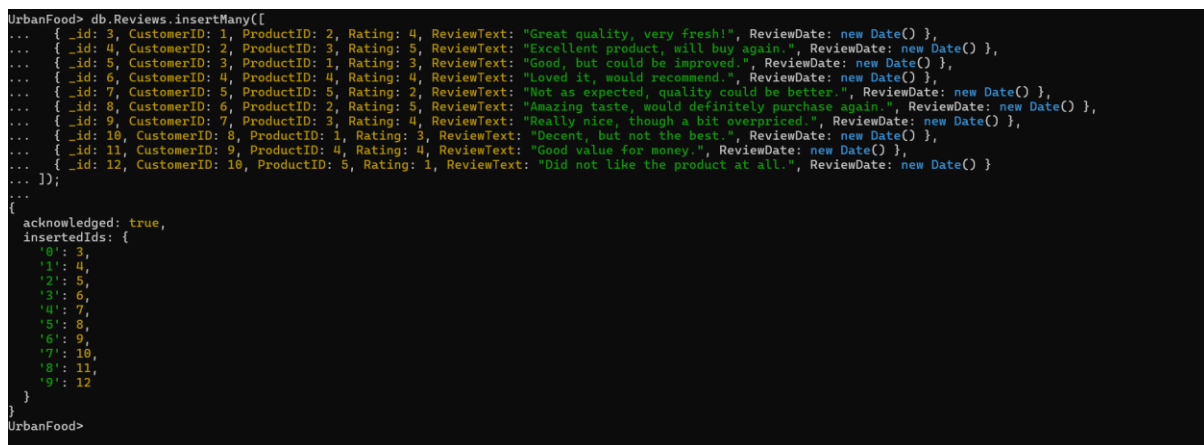
    { _id: 10, CustomerID: 8, ProductID: 1, Rating: 3, ReviewText: "Decent, but not the best.", ReviewDate: new Date() },

    { _id: 11, CustomerID: 9, ProductID: 4, Rating: 4, ReviewText: "Good value for money.", ReviewDate: new Date() },

    { _id: 12, CustomerID: 10, ProductID: 5, Rating: 1, ReviewText: "Did not like the product at all.", ReviewDate: new Date() }

]);

```



```

UrbanFood> db.Reviews.insertMany([
...   { _id: 3, CustomerID: 1, ProductID: 2, Rating: 4, ReviewText: "Great quality, very fresh!", ReviewDate: new Date() },
...   { _id: 4, CustomerID: 2, ProductID: 3, Rating: 5, ReviewText: "Excellent product, will buy again.", ReviewDate: new Date() },
...   { _id: 5, CustomerID: 3, ProductID: 1, Rating: 3, ReviewText: "Good, but could be improved.", ReviewDate: new Date() },
...   { _id: 6, CustomerID: 4, ProductID: 4, Rating: 4, ReviewText: "Loved it, would recommend.", ReviewDate: new Date() },
...   { _id: 7, CustomerID: 5, ProductID: 5, Rating: 2, ReviewText: "Not as expected, quality could be better.", ReviewDate: new Date() },
...   { _id: 8, CustomerID: 6, ProductID: 2, Rating: 5, ReviewText: "Amazing taste, would definitely purchase again.", ReviewDate: new Date() },
...   { _id: 9, CustomerID: 7, ProductID: 3, Rating: 4, ReviewText: "Really nice, though a bit overpriced.", ReviewDate: new Date() },
...   { _id: 10, CustomerID: 8, ProductID: 1, Rating: 3, ReviewText: "Decent, but not the best.", ReviewDate: new Date() },
...   { _id: 11, CustomerID: 9, ProductID: 4, Rating: 4, ReviewText: "Good value for money.", ReviewDate: new Date() },
...   { _id: 12, CustomerID: 10, ProductID: 5, Rating: 1, ReviewText: "Did not like the product at all.", ReviewDate: new Date() }
... ]);
...
{
  acknowledged: true,
  insertedIds: {
    '0': 3,
    '1': 4,
    '2': 5,
    '3': 6,
    '4': 7,
    '5': 8,
    '6': 9,
    '7': 10,
    '8': 11,
    '9': 12
  }
}
UrbanFood>

```

Figure 77-Insert many for reviews

View reviews

db.Reviews.find()

```
UrbanFood> db.Reviews.find()
[
  {
    _id: 1,
    CustomerID: 1,
    ProductID: 1,
    Rating: 5,
    ReviewText: 'Excellent quality!',
    ReviewDate: ISODate('2025-03-31T02:39:34.711Z')
  },
  {
    _id: 2,
    CustomerID: 2,
    ProductID: 2,
    Rating: 4,
    ReviewText: 'Tasty and fresh!',
    ReviewDate: ISODate('2025-03-31T02:39:34.711Z')
  },
  {
    _id: 3,
    CustomerID: 1,
    ProductID: 2,
    Rating: 4,
    ReviewText: 'Great quality, very fresh!',
    ReviewDate: ISODate('2025-03-31T02:48:08.834Z')
  },
  {
    _id: 4,
    CustomerID: 2,
    ProductID: 3,
    Rating: 5,
    ReviewText: 'Excellent product, will buy again.',
    ReviewDate: ISODate('2025-03-31T02:48:08.834Z')
  },
  {
    _id: 5,
    CustomerID: 3,
    ProductID: 1,
    Rating: 3,
    ReviewText: 'Good, but could be improved.',
  },
]
```

Figure 78-View reviews

Insert one for feedbacks

```
db.Feedbacks.insertOne({
  _id: 3,
  OrderID: 3,
  CustomerID: 3,
  FeedbackText: "The product was good, but the delivery was delayed.",
  FeedbackDate: new Date()
});
```

```
UrbanFood> db.Feedbacks.insertOne({
...   _id: 3,
...   OrderID: 3,
...   CustomerID: 3,
...   FeedbackText: "The product was good, but the delivery was delayed.",
...   FeedbackDate: new Date()
... });
...
{ acknowledged: true, insertedId: 3 }
```

Figure 79-Insert one for feedbacks

Find Rating that Rating number is 4

```
db.Reviews.find({ Rating: 4 });\
```

```
UrbanFood> db.Reviews.find({ Rating: 4 });
[
  {
    _id: 2,
    CustomerID: 2,
    ProductID: 2,
    Rating: 4,
    ReviewText: 'Tasty and fresh!',
    ReviewDate: ISODate('2025-03-31T02:39:34.711Z')
  },
  {
    _id: 3,
    CustomerID: 1,
    ProductID: 2,
    Rating: 4,
    ReviewText: 'Great quality, very fresh!',
    ReviewDate: ISODate('2025-03-31T02:48:08.834Z')
  },
  {
    _id: 6,
    CustomerID: 4,
    ProductID: 4,
    Rating: 4,
    ReviewText: 'Loved it, would recommend.',
    ReviewDate: ISODate('2025-03-31T02:48:08.834Z')
  },
  {
    _id: 9,
    CustomerID: 7,
    ProductID: 3,
    Rating: 4,
    ReviewText: 'Really nice, though a bit overpriced.',
    ReviewDate: ISODate('2025-03-31T02:48:08.834Z')
  },
  {
    _id: 11,
    CustomerID: 9,
    ProductID: 4,
    Rating: 4,
    ReviewText: 'Good value for money.',
    ReviewDate: ISODate('2025-03-31T02:48:08.834Z')
  }
]
UrbanFood>
```

Figure 80-Find Rating that Rating number is 4

Find rating that Rating numbers are 3 and 4

```
db.Reviews.find({ Rating: { $in: [3, 4] } });
```

```
UrbanFood> db.Reviews.find({ Rating: { $in: [3, 4] } });
[
  {
    _id: 2,
    CustomerID: 2,
    ProductID: 2,
    Rating: 4,
    ReviewText: 'Tasty and fresh!',
    ReviewDate: ISODate('2025-03-31T02:39:34.711Z')
  },
  {
    _id: 3,
    CustomerID: 1,
    ProductID: 2,
    Rating: 4,
    ReviewText: 'Great quality, very fresh!',
    ReviewDate: ISODate('2025-03-31T02:48:08.834Z')
  },
  {
    _id: 5,
    CustomerID: 3,
    ProductID: 1,
    Rating: 3,
    ReviewText: 'Good, but could be improved.',
    ReviewDate: ISODate('2025-03-31T02:48:08.834Z')
  }
]
```

Figure 81-Find rating that Rating numbers are 3 and 4

Find ratings less than 4

```
db.Reviews.find({ Rating: { $lt: 4 } });
```

```
UrbanFood> db.Reviews.find({ Rating: { $lt: 4 } });
[
  {
    _id: 5,
    CustomerID: 3,
    ProductID: 1,
    Rating: 3,
    ReviewText: 'Good, but could be improved.',
    ReviewDate: ISODate('2025-03-31T02:48:08.834Z')
  },
  {
    _id: 7,
    CustomerID: 5,
    ProductID: 5,
    Rating: 2,
    ReviewText: 'Not as expected, quality could be better.',
    ReviewDate: ISODate('2025-03-31T02:48:08.834Z')
  },
  {
    _id: 10,
    CustomerID: 8,
    ProductID: 1,
    Rating: 3,
    ReviewText: 'Decent, but not the best.',
    ReviewDate: ISODate('2025-03-31T02:48:08.834Z')
  },
  {
    _id: 12,
    CustomerID: 10,
    ProductID: 5,
    Rating: 1,
    ReviewText: 'Did not like the product at all.',
    ReviewDate: ISODate('2025-03-31T02:48:08.834Z')
  }
]
UrbanFood>
```

Figure 82-Find ratings less than 4

Find reviews where the rating is less than 4 or the review text contains "good"

```
db.Reviews.find({
  $or: [
    { Rating: { $lt: 4 } },
    { ReviewText: /good/i }
  ]
});
```

```
UrbanFood> db.Reviews.find({
... $or: [
...   { Rating: { $lt: 4 } },
...   { ReviewText: /good/i }
... ]
... });
[
  {
    _id: 5,
    CustomerID: 3,
    ProductID: 1,
    Rating: 3,
    ReviewText: 'Good, but could be improved.',
    ReviewDate: ISODate('2025-03-31T02:48:08.834Z')
  },
  {
    _id: 7,
    CustomerID: 5,
    ProductID: 5,
    Rating: 2,
    ReviewText: 'Not as expected, quality could be better.',
    ReviewDate: ISODate('2025-03-31T02:48:08.834Z')
  },
  {
    _id: 10,
    CustomerID: 8,
    ProductID: 1,
    Rating: 3,
    ReviewText: 'Decent, but not the best.',
    ReviewDate: ISODate('2025-03-31T02:48:08.834Z')
  },
  {
    _id: 11,
    CustomerID: 9,
    ProductID: 4,
    Rating: 4,
    ReviewText: 'Good value for money.',
    ReviewDate: ISODate('2025-03-31T02:48:08.834Z')
  },
  {
    _id: 12,
    CustomerID: 10,
    ProductID: 5,
    Rating: 1,
    ReviewText: 'Did not like the product at all.',
    ReviewDate: ISODate('2025-03-31T02:48:08.834Z')
  }
]
```

Figure 83-Find reviews where the rating is less than 4 or the review text contains "good"

Find reviews where the rating is greater than 2 and the review text contains "good"

```
db.Reviews.find({  
  
  $and: [  
  
    { Rating: { $gt: 2 } },  
  
    { ReviewText: /good/i }  
  
  ]  
  
});
```

```
UrbanFood> db.Reviews.find({  
...   $and: [  
...     { Rating: { $gt: 2 } },  
...     { ReviewText: /good/i }  
...   ]  
... });  
...  
[  
  {  
    _id: 5,  
    CustomerID: 3,  
    ProductID: 1,  
    Rating: 3,  
    ReviewText: 'Good, but could be improved.',  
    ReviewDate: ISODate('2025-03-31T02:48:08.834Z')  
  },  
  {  
    _id: 11,  
    CustomerID: 9,  
    ProductID: 4,  
    Rating: 4,  
    ReviewText: 'Good value for money.',  
    ReviewDate: ISODate('2025-03-31T02:48:08.834Z')  
  }  
]  
UrbanFood>
```

Figure 84-Find reviews where the rating is greater than 2 and the review text contains "good"

Update One feedback document

```
db.Feedbacks.updateOne(  
  
  { _id: 4 },  
  
  { $set: { FeedbackText: "Updated feedback text" } }  
  
);
```

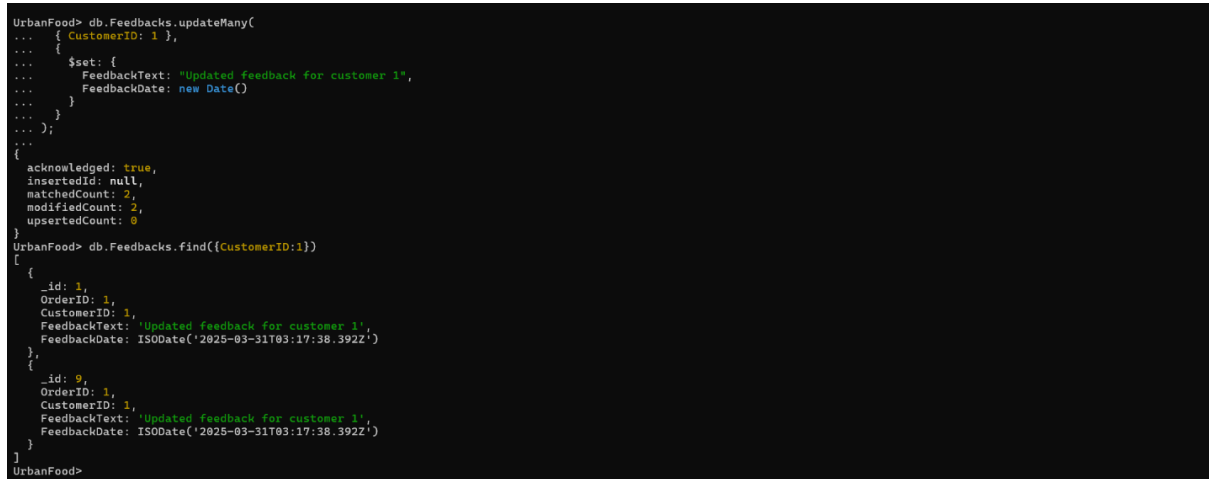
```
UrbanFood> db.Feedbacks.updateOne(  
...   { _id: 4 },  
...   { $set: { FeedbackText: "Updated feedback text" } }  
... );  
...  
{  
  acknowledged: true,  
  insertedId: null,  
  matchedCount: 1,  
  modifiedCount: 1,  
  upsertedCount: 0  
}  
UrbanFood> db.Feedbacks.find({_id:4})
```

Figure 85-Update One feedback document

```

db.Feedbacks.updateMany(
  { CustomerID: 1 },
  {
    $set: {
      FeedbackText: "Updated feedback for customer 1",
      FeedbackDate: new Date()
    }
  }
);

```



```

UrbanFood> db.Feedbacks.updateMany(
... { CustomerID: 1 },
... {
...   $set: {
...     FeedbackText: "Updated feedback for customer 1",
...     FeedbackDate: new Date()
...   }
... }
... );
...
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 2,
  modifiedCount: 2,
  upsertedCount: 0
}
UrbanFood> db.Feedbacks.find({CustomerID:1})
[
  {
    _id: 1,
    OrderID: 1,
    CustomerID: 1,
    FeedbackText: 'Updated feedback for customer 1',
    FeedbackDate: ISODate('2025-03-31T03:17:38.392Z')
  },
  {
    _id: 9,
    OrderID: 1,
    CustomerID: 1,
    FeedbackText: 'Updated feedback for customer 1',
    FeedbackDate: ISODate('2025-03-31T03:17:38.392Z')
  }
]
UrbanFood>

```

Figure 86-Update One feedback document

Replace One document from reviews

```

db.Reviews.replaceOne(
  { _id: 4 },
  {
    _id: 4,
    CustomerID: 1,
    ProductID: 2,
    Rating: 5,

```

```

    ReviewText: "This is an updated review with a new rating.",
    ReviewDate: new Date()
  }
);

```

```

UrbanFood> db.Reviews.replaceOne(
... { _id: 4 },
... {
...   _id: 4,
...   CustomerID: 1,
...   ProductID: 2,
...   Rating: 5,
...   ReviewText: "This is an updated review with a new rating.",
...   ReviewDate: new Date()
... }
... );
...
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
UrbanFood>

```

Figure 87-Replace One document from reviews

Delete one feedback document where id is 8

```
db.Feedbacks.deleteOne({ _id: 8 });
```

```

UrbanFood> db.Feedbacks.deleteOne({ _id: 8 });
{ acknowledged: true, deletedCount: 1 }
UrbanFood>

```

Figure 88-Delete one feedback document where id is 8

Delete Many which is rating number is one

```
db.Reviews.deleteMany({ Rating: 1 });
```

```

UrbanFood> db.Reviews.deleteMany({ Rating: 1 });
{ acknowledged: true, deletedCount: 1 }
UrbanFood>

```

Figure 89-Delete Many which is rating number is one

References

GitHub Link:- <https://github.com/ShashimalMadhuwantha/UrbanFood-PL-SQL.git>